

# Midterm Review 2

CS 185/285

Instructor: Sergey Levine  
UC Berkeley



# Part 4:

## Q-learning

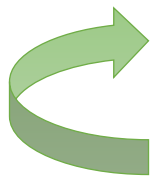
# Policy iteration with dynamic programming

policy iteration:

- 
1. evaluate  $V^\pi(\mathbf{s})$
  2. set  $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

policy evaluation:

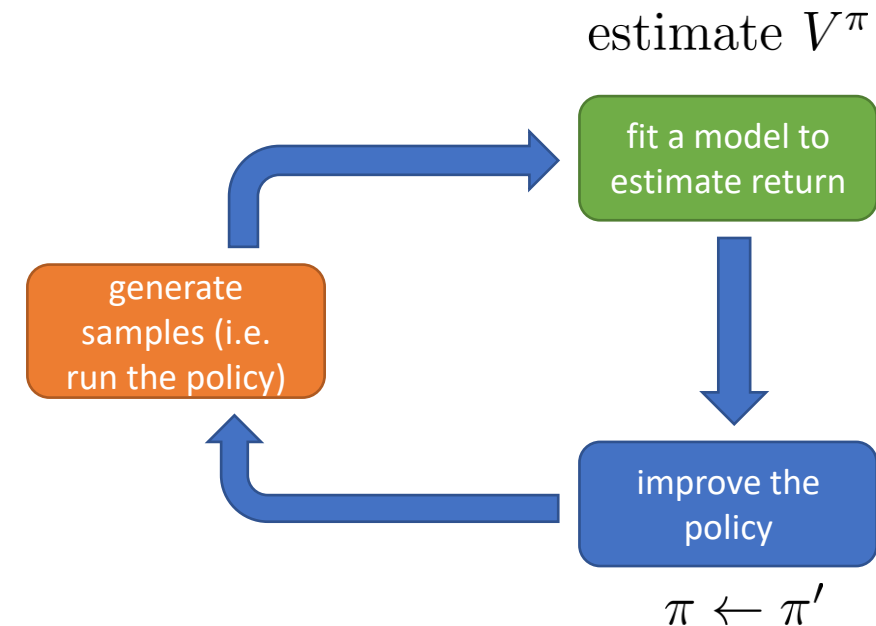

$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')] ]$$

|     |     |     |     |
|-----|-----|-----|-----|
| 0.2 | 0.3 | 0.4 | 0.3 |
| 0.3 | 0.3 | 0.5 | 0.3 |
| 0.4 | 0.4 | 0.6 | 0.4 |
| 0.5 | 0.5 | 0.7 | 0.5 |

16 states, 4 actions per state

can store full  $V^\pi(\mathbf{s})$  in a table!

$\mathcal{T}$  is  $16 \times 16 \times 4$  tensor



# Even simpler dynamic programming

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

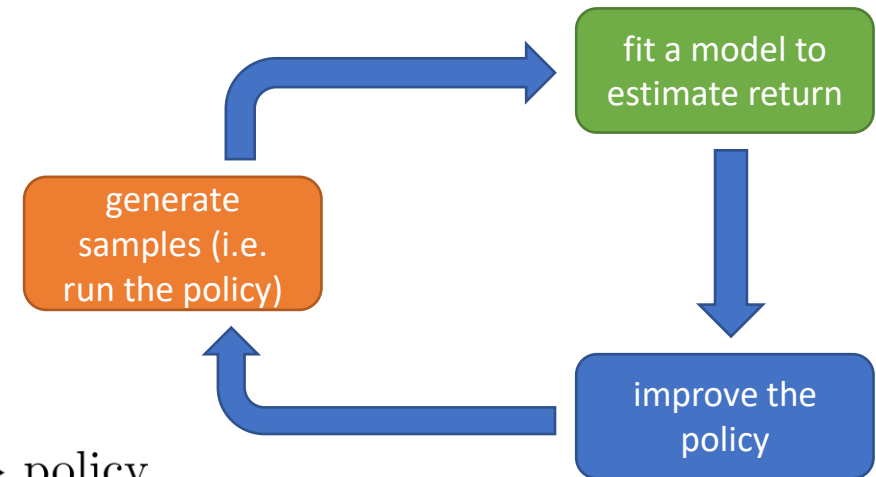
let's evaluate  $Q^\pi(\mathbf{s})$ !

|          |                             |                             |                             |                             |
|----------|-----------------------------|-----------------------------|-----------------------------|-----------------------------|
|          | <b>a</b>                    |                             |                             |                             |
| <b>s</b> | $Q(\mathbf{s}, \mathbf{a})$ |                             | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ |
|          | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ |                             | $Q(\mathbf{s}, \mathbf{a})$ |
|          |                             | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ |
|          | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ |                             |
|          | $Q(\mathbf{s}, \mathbf{a})$ |                             | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ |
|          | $Q(\mathbf{s}, \mathbf{a})$ | $Q(\mathbf{s}, \mathbf{a})$ |                             | $Q(\mathbf{s}, \mathbf{a})$ |

$\arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}) \rightarrow \text{policy}$

approximates the new value!

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^\pi(\mathbf{s}')] ]$$



$$V^\pi(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q^\pi(\mathbf{s}, \mathbf{a})$$

skip the policy and compute values directly!

value iteration algorithm:

1. set  $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')] ]$
2. set  $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

# What if we don't know the transition dynamics?

fitted value iteration algorithm:

- 
1. set  $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)])$
  2. set  $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$
- need to know outcomes for different actions!

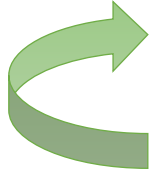
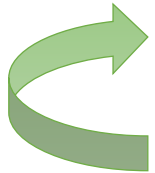
Back to policy iteration...

policy iteration:

- 
1. evaluate  $Q^\pi(\mathbf{s}, \mathbf{a})$
  2. set  $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

policy evaluation:


$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')]$$

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \mathbf{a})} [Q^\pi(\mathbf{s}', \pi(\mathbf{s}'))]$$

can fit this using samples



# Can we do the “max” trick again?

policy iteration:



1. evaluate  $V^\pi(\mathbf{s})$
2. set  $\pi \leftarrow \pi'$



fitted value iteration algorithm:



1. set  $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)])$
2. set  $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$

no policy, compute value directly

can we do this with Q-values **also**, without knowing the transitions?

fitted Q iteration algorithm:



1. set  $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)]$
2. set  $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

approximate  $E[V(\mathbf{s}'_i)] \approx \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$

doesn't require simulation of actions!

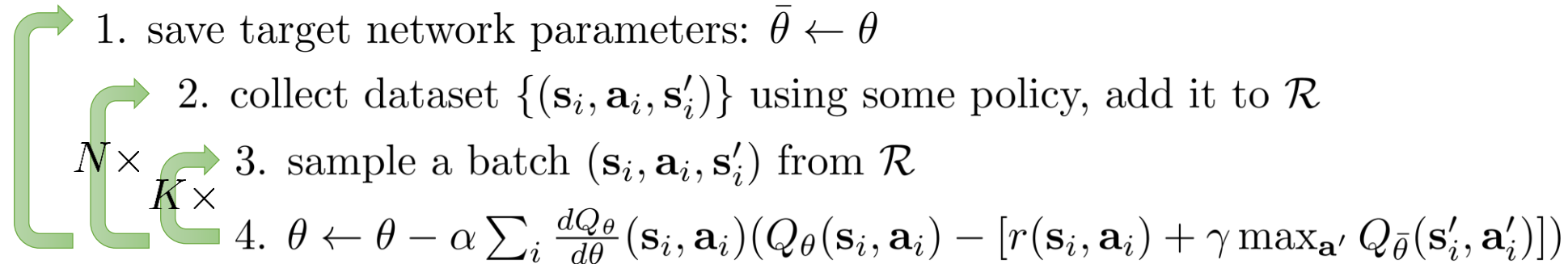
+ works even for off-policy samples

+ only one network, no high-variance policy gradient

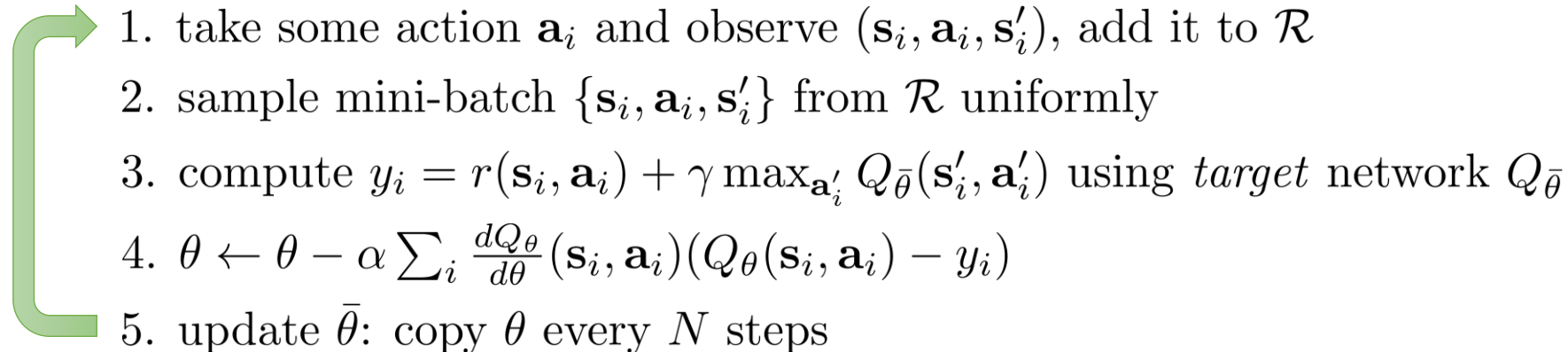
- no convergence guarantees for non-linear function approximation (more on this later)

# “Classic” deep Q-learning algorithm (DQN)

Generalized Q-learning with replay buffer and target network:

- 
1. save target network parameters:  $\bar{\theta} \leftarrow \theta$
  2. collect dataset  $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$  using some policy, add it to  $\mathcal{R}$
  3. sample a batch  $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$  from  $\mathcal{R}$
  4.  $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$

“classic” deep Q-learning algorithm:

- 
1. take some action  $\mathbf{a}_i$  and observe  $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ , add it to  $\mathcal{R}$
  2. sample mini-batch  $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$  from  $\mathcal{R}$  uniformly
  3. compute  $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$  using *target* network  $Q_{\bar{\theta}}$
  4.  $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i)$
  5. update  $\bar{\theta}$ : copy  $\theta$  every  $N$  steps

# Double Q-learning

$$E[\max(X_1, X_2)] \geq \max(E[X_1], E[X_2])$$

note that  $\max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}') = \underline{Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}'))}$

value *also* comes from  $Q_{\bar{\theta}}$     action selected according to  $Q_{\bar{\theta}}$

if the noise in these is decorrelated, the problem goes away!

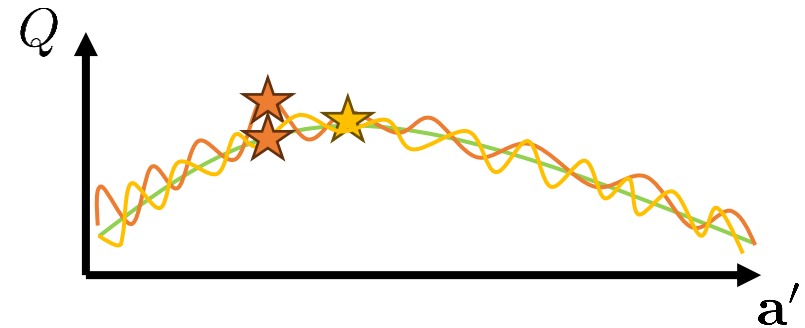
idea: don't use the same network to choose the action and evaluate value!

“double” Q-learning: use two networks:

$$Q_{\theta_A}(\mathbf{s}, \mathbf{a}) \leftarrow r + \gamma Q_{\theta_B}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta_A}(\mathbf{s}', \mathbf{a}'))$$

$$Q_{\theta_B}(\mathbf{s}, \mathbf{a}) \leftarrow r + \gamma Q_{\theta_A}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta_B}(\mathbf{s}', \mathbf{a}'))$$

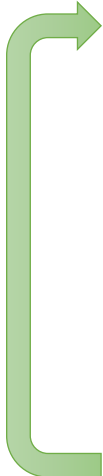
if the two Q's are noisy in *different* ways, there is no problem





# DDPG algorithm summary

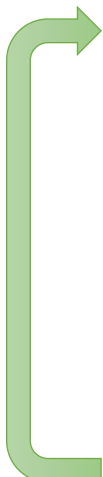
DDPG:

- 
1. take some action  $\mathbf{a}_i$  and observe  $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ , add it to  $\mathcal{R}$
  2. sample mini-batch  $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$  from  $\mathcal{R}$  uniformly
  3. compute  $y_j = r(\mathbf{s}_j, \mathbf{a}_j) + \gamma Q_{\bar{\phi}}(\mathbf{s}'_j, \mu_{\bar{\theta}}(\mathbf{s}'_j))$  using *target* nets  $Q_{\bar{\phi}}$  and  $\mu_{\bar{\theta}}$
  4.  $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_{\phi}}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_{\phi}(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
  5.  $\theta \leftarrow \theta + \beta \sum_j \frac{d\mu}{d\theta}(\mathbf{s}_j) \frac{dQ_{\phi}}{d\mathbf{a}}(\mathbf{s}_j, \mu(\mathbf{s}_j))$
  6. update  $\bar{\phi}$  and  $\bar{\theta}$  (e.g., Polyak averaging)

can improve with clipped double-Q

$$\mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [\min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(\mathbf{s}', \mathbf{a}')] ]$$

Off-policy actor-critic:

- 
1. take some action  $\mathbf{a}_i$  and observe  $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ , add it to  $\mathcal{R}$
  2. sample mini-batch  $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$  from  $\mathcal{R}$  uniformly
  3. compute  $y_j = r(\mathbf{s}_j, \mathbf{a}_j) + \gamma \mathbb{E}_{\mathbf{a}'_j \sim \pi_{\bar{\theta}}} [Q_{\bar{\phi}}(\mathbf{s}'_j, \mathbf{a}'_j)]$  using *target* nets  $Q_{\bar{\phi}}$  and  $\mu_{\bar{\theta}}$
  4.  $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_{\phi}}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_{\phi}(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
  5.  $\theta \leftarrow \theta + \beta \sum_j \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_j|\mathbf{s}_j) Q_{\phi}(\mathbf{s}_j, \mathbf{a}_j)$
  6. update  $\bar{\phi}$  and  $\bar{\theta}$  (e.g., Polyak averaging)

What are some other improvements we learned about?

# Non-tabular value function learning

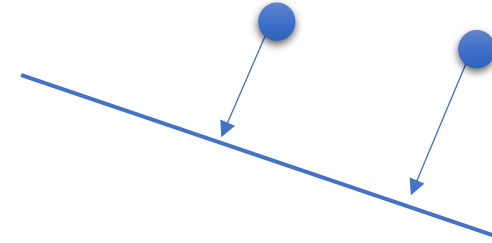
fitted value iteration algorithm (using  $\mathcal{B}$  and  $\Pi$ ):

1.  $V \leftarrow \Pi \mathcal{B} V$

$\mathcal{B}$  is a contraction w.r.t.  $\infty$ -norm (“max” norm)

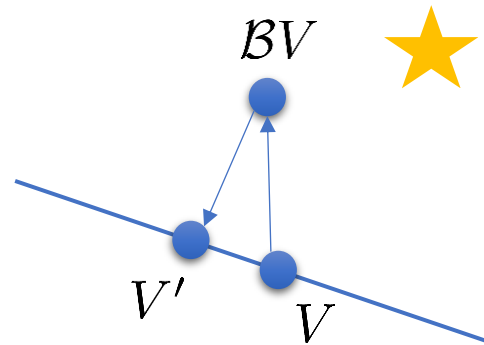
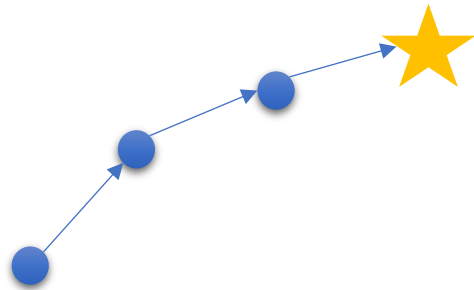
$\Pi$  is a contraction w.r.t.  $\ell_2$ -norm (Euclidean distance)

but...  $\Pi \mathcal{B}$  is not a contraction of any kind



$$\|\mathcal{B}V - \mathcal{B}\bar{V}\|_{\infty} \leq \gamma \|V - \bar{V}\|_{\infty}$$

$$\|\Pi V - \Pi \bar{V}\|^2 \leq \|V - \bar{V}\|^2$$



Conclusions:  
value iteration converges  
(tabular case)  
fitted value iteration does **not**  
converge  
not in general  
often not in practice

# Part 5:

## Variational inference

# The variational approximation

but... how do we calculate  $p(z|x_i)$ ?

can bound  $\log p(x_i)$ !

$$\log p(x_i) = \log \int_z p(x_i|z)p(z)$$

$$= \log \int_z p(x_i|z)p(z) \frac{q_i(z)}{q_i(z)}$$

$$= \log E_{z \sim q_i(z)} \left[ \frac{p(x_i|z)p(z)}{q_i(z)} \right]$$

$$\geq E_{z \sim q_i(z)} \left[ \log \frac{p(x_i|z)p(z)}{q_i(z)} \right] = E_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] + \mathbf{E}_{z \sim q_i(z)} [\log q_i(z)]$$

maximizing this maximizes  $\log p(x_i)$



Jensen's inequality

$$\log E[y] \geq E[\log y]$$

# The variational approximation

$$\mathcal{L}_i(p, q_i)$$

$$\log p(x_i) \geq \overbrace{E_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)]}^{\mathcal{L}_i(p, q_i)} + \mathcal{H}(q_i)$$

what makes a good  $q_i(z)$ ?

intuition:  $q_i(z)$  should approximate  $p(z|x_i)$

approximate in what sense?

compare in terms of KL-divergence:  $D_{\text{KL}}(q_i(z) \| p(z|x))$

why?

$$\begin{aligned} D_{\text{KL}}(q_i(x_i) \| p(z|x_i)) &= E_{z \sim q_i(z)} \left[ \log \frac{q_i(z)}{p(z|x_i)} \right] = E_{z \sim q_i(z)} \left[ \log \frac{q_i(z)p(x_i)}{p(x_i, z)} \right] \\ &= -E_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] + E_{z \sim q_i(z)} [\log q_i(z)] + E_{z \sim q_i(z)} [\log p(x_i)] \\ &= -E_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] - \mathcal{H}(q_i) + \log p(x_i) \\ &= -\mathcal{L}_i(p, q_i) + \log p(x_i) \end{aligned}$$

$$\log p(x_i) = D_{\text{KL}}(q_i(z) \| p(z|x_i)) + \mathcal{L}_i(p, q_i)$$

$$\log p(x_i) \geq \mathcal{L}_i(p, q_i)$$

# The variational approximation

$$\log p(x_i) \geq \overbrace{E_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)]}^{\mathcal{L}_i(p, q_i)} + \mathcal{H}(q_i)$$

$$\log p(x_i) = D_{\text{KL}}(q_i(z) \| p(z|x_i)) + \mathcal{L}_i(p, q_i)$$

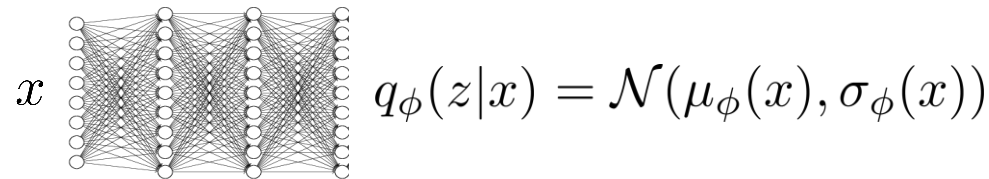
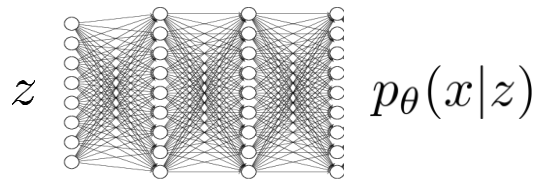
$$\log p(x_i) \geq \mathcal{L}_i(p, q_i)$$

$$\begin{aligned} D_{\text{KL}}(q_i(z) \| p(z|x_i)) &= E_{z \sim q_i(z)} \left[ \log \frac{q_i(z)}{p(z|x_i)} \right] = E_{z \sim q_i(z)} \left[ \log \frac{q_i(z)p(x_i)}{p(x_i, z)} \right] \\ &= \underbrace{-E_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)]}_{-\mathcal{L}_i(p, q_i)} - \mathcal{H}(q_i) + \log p(x_i) \end{aligned}$$

independent of  $q_i$ !

$\Rightarrow$  maximizing  $\mathcal{L}_i(p, q_i)$  w.r.t.  $q_i$  minimizes KL-divergence!

# Amortized variational inference



for each  $x_i$  (or mini-batch):

calculate  $\nabla_\theta \mathcal{L}(p_\theta(x_i|z), q_\phi(z|x_i))$ :

sample  $z \sim q_\phi(z|x_i)$

$\nabla_\theta \mathcal{L} \approx \nabla_\theta \log p_\theta(x_i|z)$

$\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$

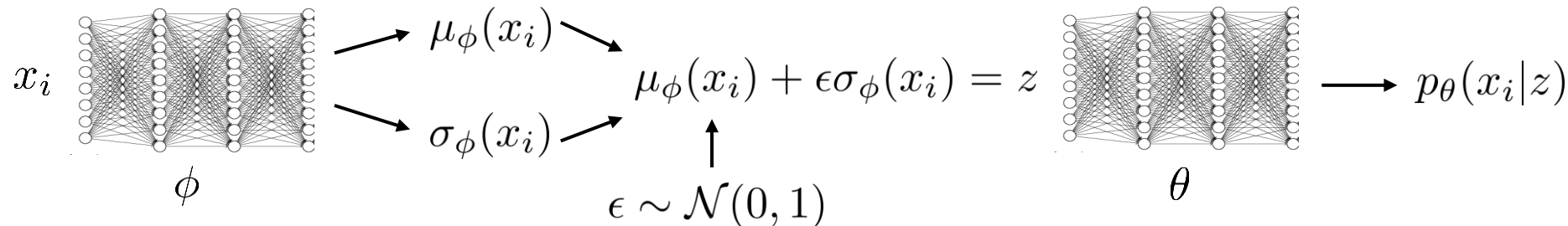
$\phi \leftarrow \phi + \alpha \nabla_\phi \mathcal{L}$

how do we calculate this?

$$\log p(x_i) \geq \overbrace{E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z) + \log p(z)]}^{\mathcal{L}(p_\theta(x_i|z), q_\phi(z|x_i))} + \mathcal{H}(q_\phi(z|x_i))$$

# Another way to look at it...

$$\begin{aligned}\mathcal{L}_i &= E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z) + \log p(z)] + \mathcal{H}(q_\phi(z|x_i)) \\ &= E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z)] + \underbrace{E_{z \sim q_\phi(z|x_i)} [\log p(z)] + \mathcal{H}(q_\phi(z|x_i))}_{-D_{\text{KL}}(q_\phi(z|x_i) \| p(z))} \leftarrow \text{this often has a convenient analytical form (e.g., KL-divergence for Gaussians)} \\ &= E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z)] - D_{\text{KL}}(q_\phi(z|x_i) \| p(z)) \\ &= E_{\epsilon \sim \mathcal{N}(0,1)} [\log p_\theta(x_i | \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i))] - D_{\text{KL}}(q_\phi(z|x_i) \| p(z)) \\ &\approx \log p_\theta(x_i | \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i)) - D_{\text{KL}}(q_\phi(z|x_i) \| p(z))\end{aligned}$$





# Conditional models

$$\mathcal{L}_i = E_{z \sim q_\phi(z|x_i, y_i)} [\log p_\theta(y_i|x_i, z) + \log p(z|x_i)] + \mathcal{H}(q_\phi(z|x_i, y_i))$$

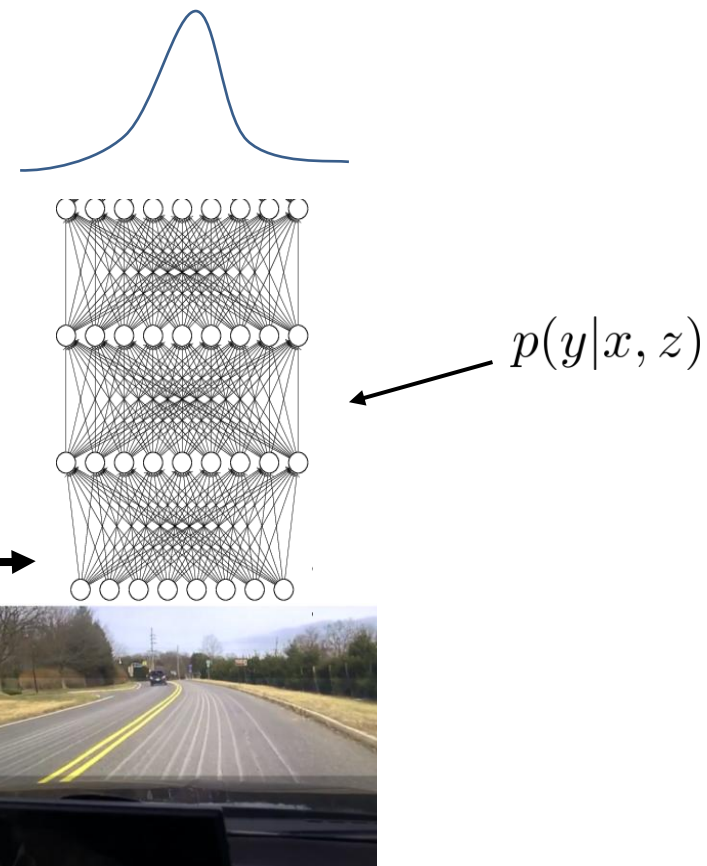
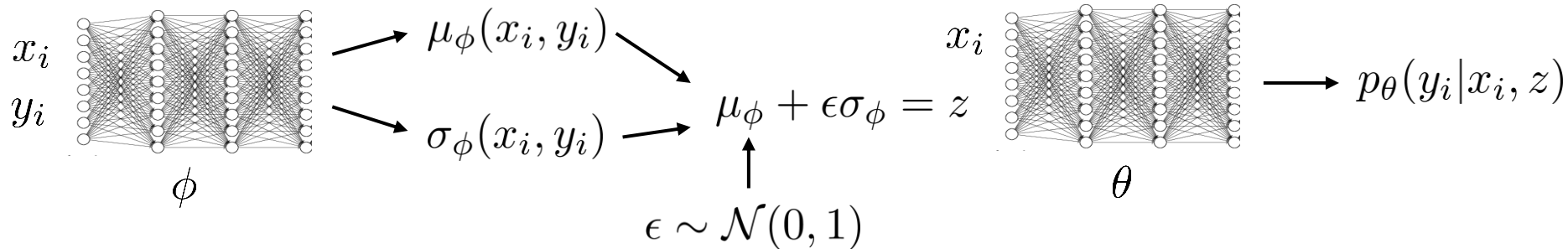
just like before, only now generating  $y_i$   
and *everything* is conditioned on  $x_i$

at test time:

$$z \sim p(z|x_i)$$

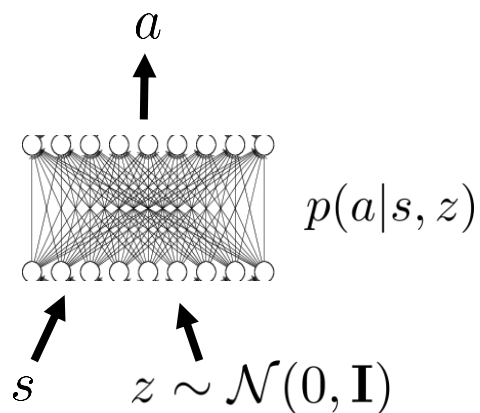
$$y \sim p(y|x_i, z)$$

can *optionally* depend on  $x$



# Relationship to diffusion and flow matching

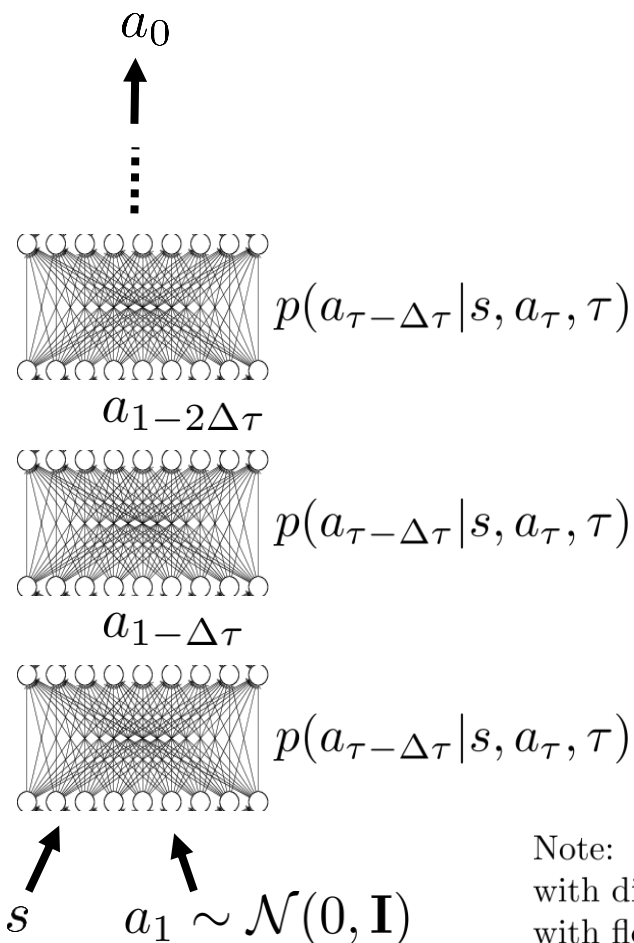
conditional VAE



$$\log p(a|s) \geq$$

$$E_{z \sim q(z|s)} [\log p(a|s, z)] - D_{\text{KL}}(q(z|s), p(z))$$

diffusion or flow



$$\log p(a_0|s) \geq$$

$$E_{a_{\Delta\tau:1} \sim q(a_{\Delta\tau:1}|s)} [\log p(a_0|s, a_{\Delta\tau:1})]$$

$$-D_{\text{KL}}(q(a_{\Delta\tau:1}|a_0), p(a_{\Delta\tau:1}))$$

forward  
(noising)  
process

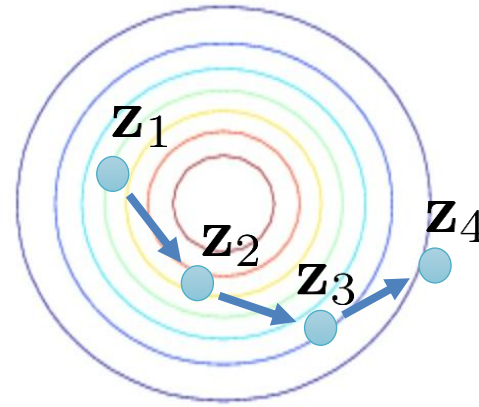
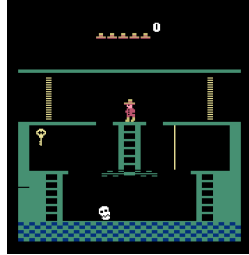
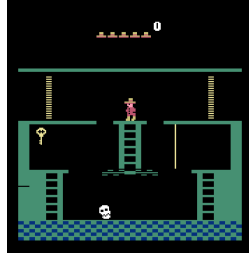
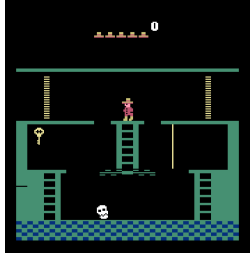
reverse  
(denoising)  
process

diffusion can be thought of  
as a hierarchical VAE

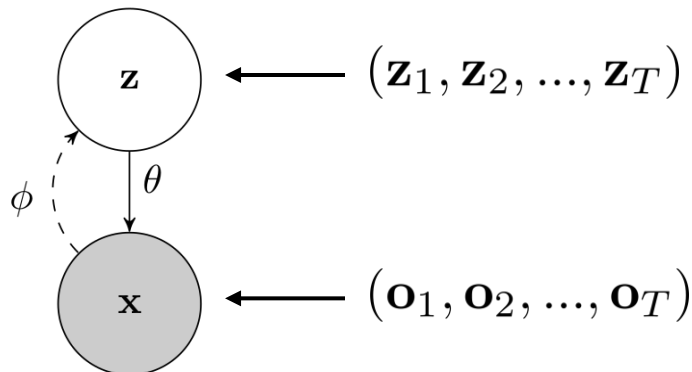
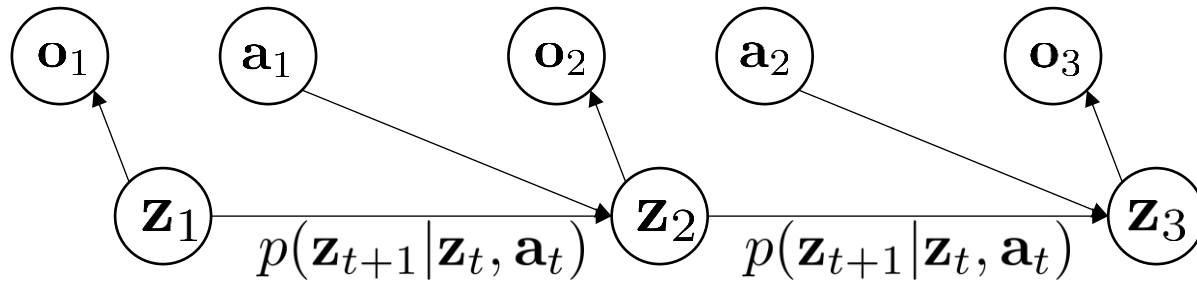
Note:

with diffusion (DDPM), noise is  $a_1$   
with flow matching, noise is  $a_0$

# State space models



we'll learn more about this when we discuss model-based RL!



What is the **prior**?

What is the **decoder**?

What is the **encoder**?

$$p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$$

$\mathcal{N}(0, \mathbf{I})$  (prior) points to  $p(\mathbf{z}_1)$   
 learned points to  $p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$

$$p_\theta(\mathbf{o} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{z}_t)$$

$$q_\phi(\mathbf{z} | \mathbf{o}) = \prod_t q_\phi(\mathbf{z}_t | \mathbf{o}_{1:t})$$

# Part 6:

## Control as inference & IRL

# A probabilistic graphical model of decision making

~~$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)$$

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$~~

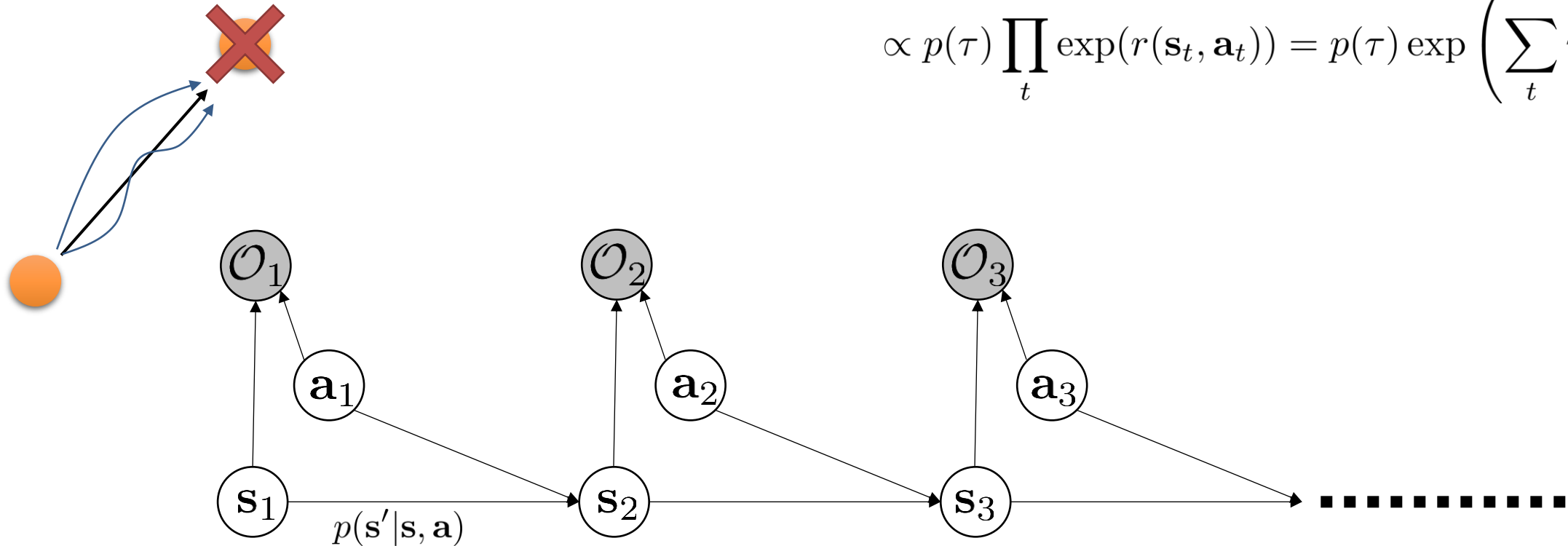
$$p(\underbrace{\mathbf{s}_{1:T}, \mathbf{a}_{1:T}}_{\tau}) = ?? \quad \text{no assumption of optimal behavior!}$$

$$p(\tau | \mathcal{O}_{1:T})$$

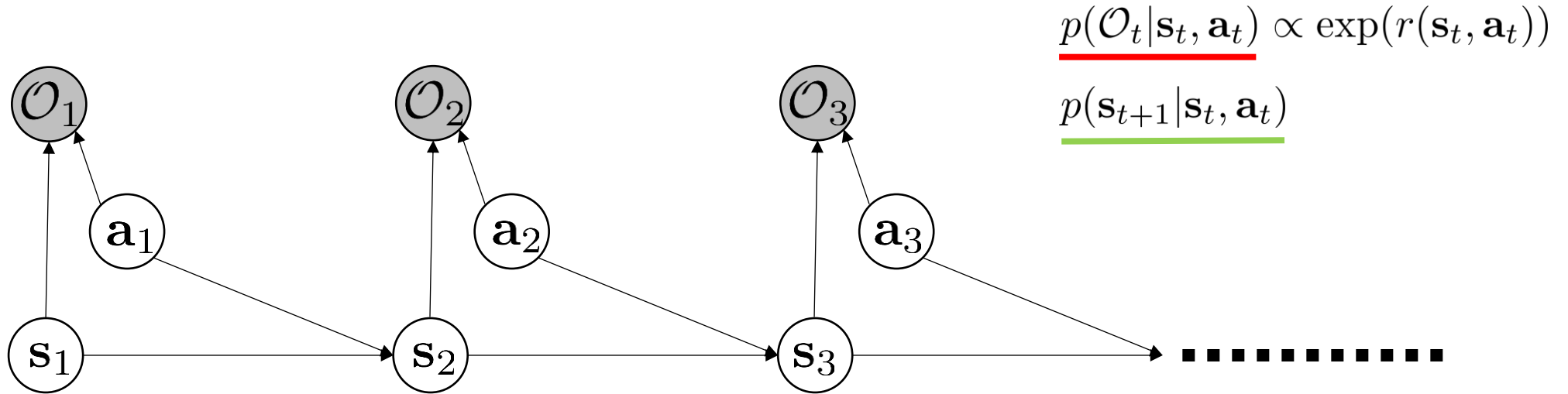
$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$$

$$p(\tau | \mathcal{O}_{1:T}) = \frac{p(\tau, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})}$$

$$\propto p(\tau) \prod_t \exp(r(\mathbf{s}_t, \mathbf{a}_t)) = p(\tau) \exp\left(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right)$$



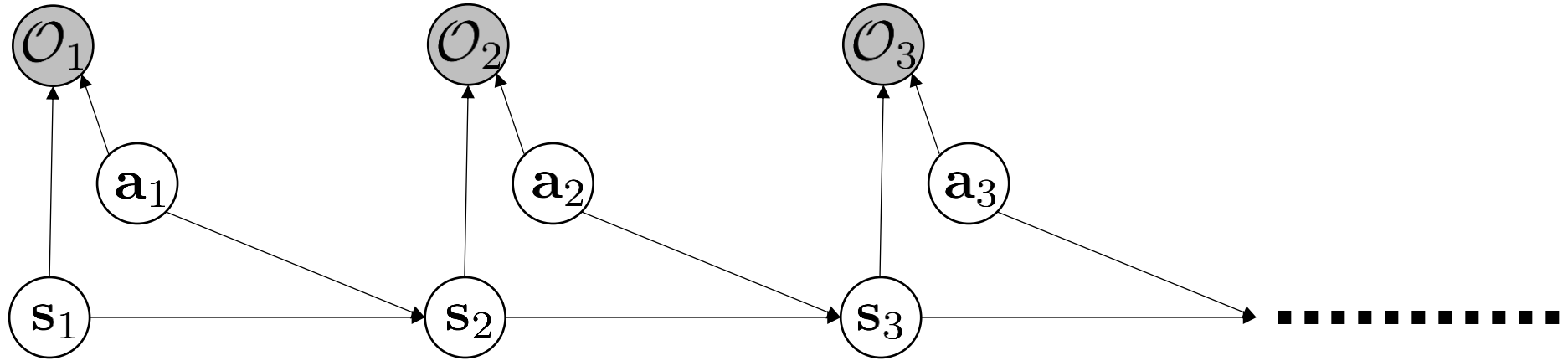
# Backward messages



$$\begin{aligned}
 \beta_t(\mathbf{s}_t, \mathbf{a}_t) &= p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t) \\
 &= \int p(\mathcal{O}_{t:T}, \mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) d\mathbf{s}_{t+1} \\
 &= \int p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}) \underbrace{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{\text{green}} \underbrace{p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t)}_{\text{red}} d\mathbf{s}_{t+1} \quad \text{for } t = T - 1 \text{ to } 1: \\
 &\quad \longrightarrow \beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})] \\
 p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}) &= \int \underbrace{p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}, \mathbf{a}_{t+1})}_{\beta_t(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})} \underbrace{p(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})}_{\text{red, crossed out}} d\mathbf{a}_{t+1} \\
 &\quad \longrightarrow \beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)]
 \end{aligned}$$

which actions are likely *a priori*  
 (assume uniform for now)

# Backward pass summary



$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t)$$

probability that we can be optimal at steps  $t$  through  $T$   
given that we take action  $\mathbf{a}_t$  in state  $\mathbf{s}_t$

for  $t = T - 1$  to 1:

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})] \quad \text{compute recursively from } t = T \text{ to } t = 1$$

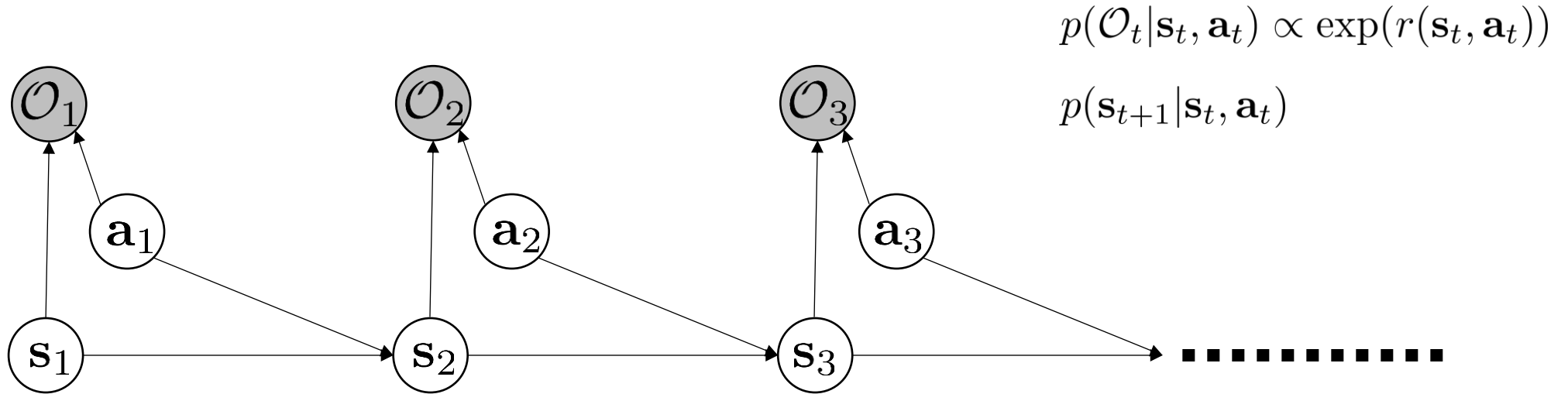
$$\beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)]$$

$$\text{let } V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t)$$

$$\text{let } Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t)$$

log of  $\beta_t$  is “ $Q$ -function-like”

# Policy computation



2. compute policy  $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T})$

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\beta_t(\mathbf{s}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t)$$

$$p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}) = \pi(\mathbf{a}_t | \mathbf{s}_t)$$

$$= p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{t:T})$$

$$= \frac{p(\mathbf{a}_t, \mathbf{s}_t | \mathcal{O}_{t:T})}{p(\mathbf{s}_t | \mathcal{O}_{t:T})}$$

$$= \frac{p(\mathcal{O}_{t:T} | \mathbf{a}_t, \mathbf{s}_t) p(\mathbf{a}_t, \mathbf{s}_t) / \cancel{p(\mathcal{O}_{t:T})}}{p(\mathcal{O}_{t:T} | \mathbf{s}_t) p(\mathbf{s}_t) / \cancel{p(\mathcal{O}_{t:T})}}$$

$$= \frac{p(\mathcal{O}_{t:T} | \mathbf{a}_t, \mathbf{s}_t)}{p(\mathcal{O}_{t:T} | \mathbf{s}_t)} \frac{p(\mathbf{a}_t, \mathbf{s}_t)}{p(\mathbf{s}_t)} = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} \cancel{p(\mathbf{a}_t | \mathbf{s}_t)}$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)}$$



# Policy computation with value functions

for  $t = T - 1$  to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log E[\exp(V_{t+1}(\mathbf{s}_{t+1}))]$$

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) \mathbf{a}_t$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} \quad \begin{array}{l} V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t) \\ Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t) \end{array}$$

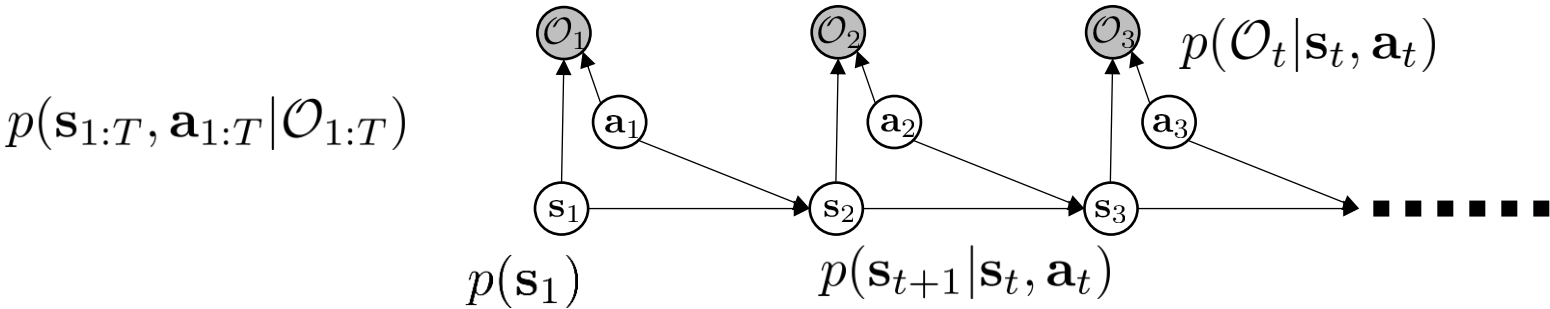
$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)) = \exp(A_t(\mathbf{s}_t, \mathbf{a}_t))$$

# Control via variational inference

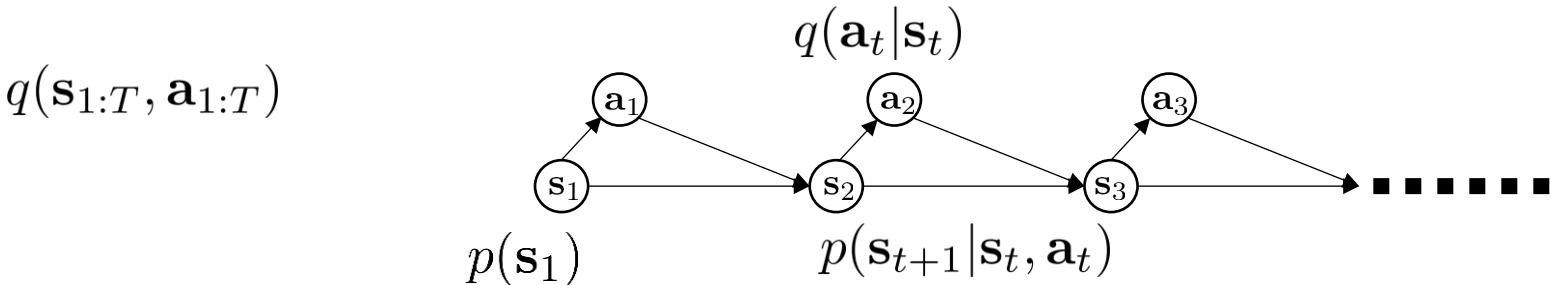
let  $q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) q(\mathbf{a}_t | \mathbf{s}_t)$

same dynamics and initial state as  $p$       only new thing

let  $\mathbf{x} = \mathcal{O}_{1:T}$  and  $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$



$p(\mathbf{z} | \mathbf{x})$



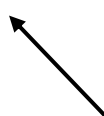
$q(\mathbf{z})$

# The variational lower bound

$$\log p(\mathbf{x}) \geq E_{\mathbf{z} \sim q(\mathbf{z})} [\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z})]$$

let  $\mathbf{x} = \mathcal{O}_{1:T}$  and  $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$

the entropy  $\mathcal{H}(q)$



$$\text{let } q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) = \underbrace{p(\mathbf{s}_1)}_{\text{green}} \underbrace{\prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{\text{orange}} q(\mathbf{a}_t | \mathbf{s}_t)$$

$$\begin{aligned} \log p(\mathcal{O}_{1:T}) \geq E_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} [ & \log p(\mathbf{s}_1) + \sum_{t=1}^T \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) + \sum_{t=1}^T \log p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) \\ & \underbrace{- \log p(\mathbf{s}_1)}_{\text{green}} - \underbrace{\sum_{t=1}^T \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{\text{orange}} - \sum_{t=1}^T \log q(\mathbf{a}_t | \mathbf{s}_t) ] \end{aligned}$$

$$= E_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) - \log q(\mathbf{a}_t | \mathbf{s}_t) \right]$$

$$= \sum_t E_{(\mathbf{s}_t, \mathbf{a}_t) \sim q} [r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(q(\mathbf{a}_t | \mathbf{s}_t))]$$

maximize reward and maximize action entropy!

# Backward pass summary - variational

for  $t = T - 1$  to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + E[(V_{t+1}(\mathbf{s}_{t+1}))]$$

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t$$

value iteration algorithm:

- 
1. set  $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')] ]$
  2. set  $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

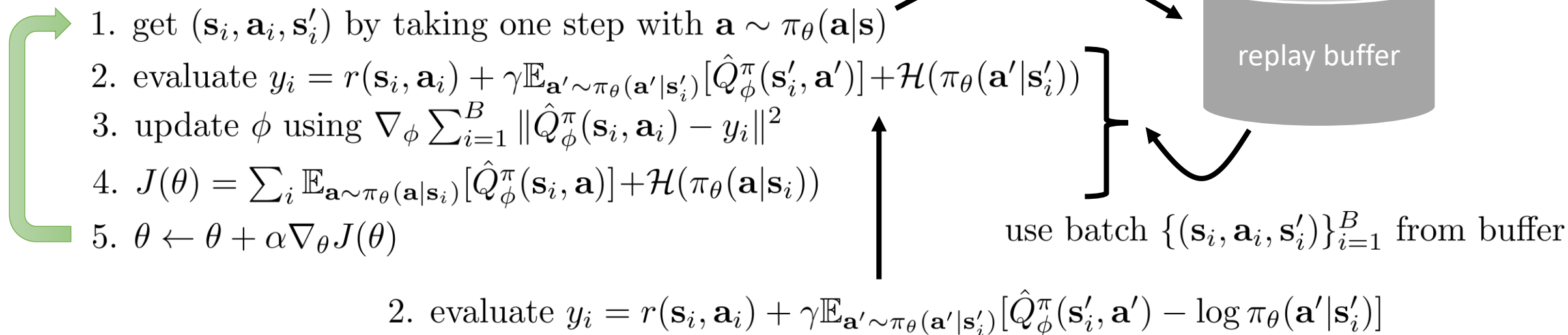
*soft* value iteration algorithm:

- 
1. set  $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')] ]$
  2. set  $V(\mathbf{s}) \leftarrow \text{soft max}_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

# Soft actor-critic

What if we want an *actor-critic* algorithm with soft optimality?

off-policy actor-critic algorithm:



This is one of the most commonly used  
off-policy actor-critic algorithms!

usually put a “temperature”  $\beta$  in front of  $\mathcal{H}(\pi_\theta(\mathbf{a}|\mathbf{s}_i))$

why?

# Learning the optimality variable

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t, \psi) \propto \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t))$$

$\psi$

$r_\psi(\mathbf{s}_t, \mathbf{a}_t)$

reward parameters

given:

samples  $\{\tau_i\}$  sampled from  $\pi^*(\tau)$

$$p(\tau | \mathcal{O}_{1:T}, \psi) \propto \cancel{p(\tau)} \exp\left(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\right)$$

can ignore (independent of  $\psi$ )

maximum likelihood learning:

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N \log p(\tau_i | \mathcal{O}_{1:T}, \psi) = \max_{\psi} \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \log Z$$

partition function  
(the hard part)

# The IRL partition function

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N r_{\psi}(\tau_i) - \log Z$$

$$Z = \int p(\tau) \exp(r_{\psi}(\tau)) d\tau$$

$$\nabla_{\psi} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \underbrace{\frac{1}{Z} \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau}_{p(\tau | \mathcal{O}_{1:T}, \psi)}$$

$$\nabla_{\psi} \mathcal{L} = E_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)] - E_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]$$



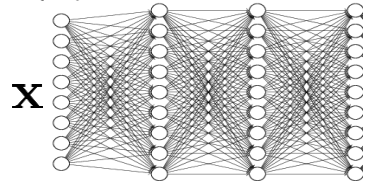
estimate with expert samples



soft optimal policy under current reward

# Inverse RL as a GAN

$$D(\mathbf{x}) = p_\psi(\text{real image}|\mathbf{x})$$



which discriminator is best?

$$D^*(\mathbf{x}) = \frac{p^*(\mathbf{x})}{p_\theta(\mathbf{x}) + p^*(\mathbf{x})}$$

for IRL, optimal policy approaches  $\pi_\theta(\tau) \propto p(\tau) \exp(r_\psi(\tau))$

choose this parameterization for discriminator:

optimize  $Z$  w.r.t. same objective as  $\psi$ !

$$D_\psi(\tau) = \frac{p(\tau) \frac{1}{Z} \exp(r(\tau))}{p_\theta(\tau) + p(\tau) \frac{1}{Z} \exp(r(\tau))} = \frac{\cancel{p(\tau)} \frac{1}{Z} \exp(r(\tau))}{\cancel{p(\tau)} \prod_t \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) + \cancel{p(\tau)} \frac{1}{Z} \exp(r(\tau))} = \frac{\frac{1}{Z} \exp(r(\tau))}{\prod_t \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) + \frac{1}{Z} \exp(r(\tau))}$$

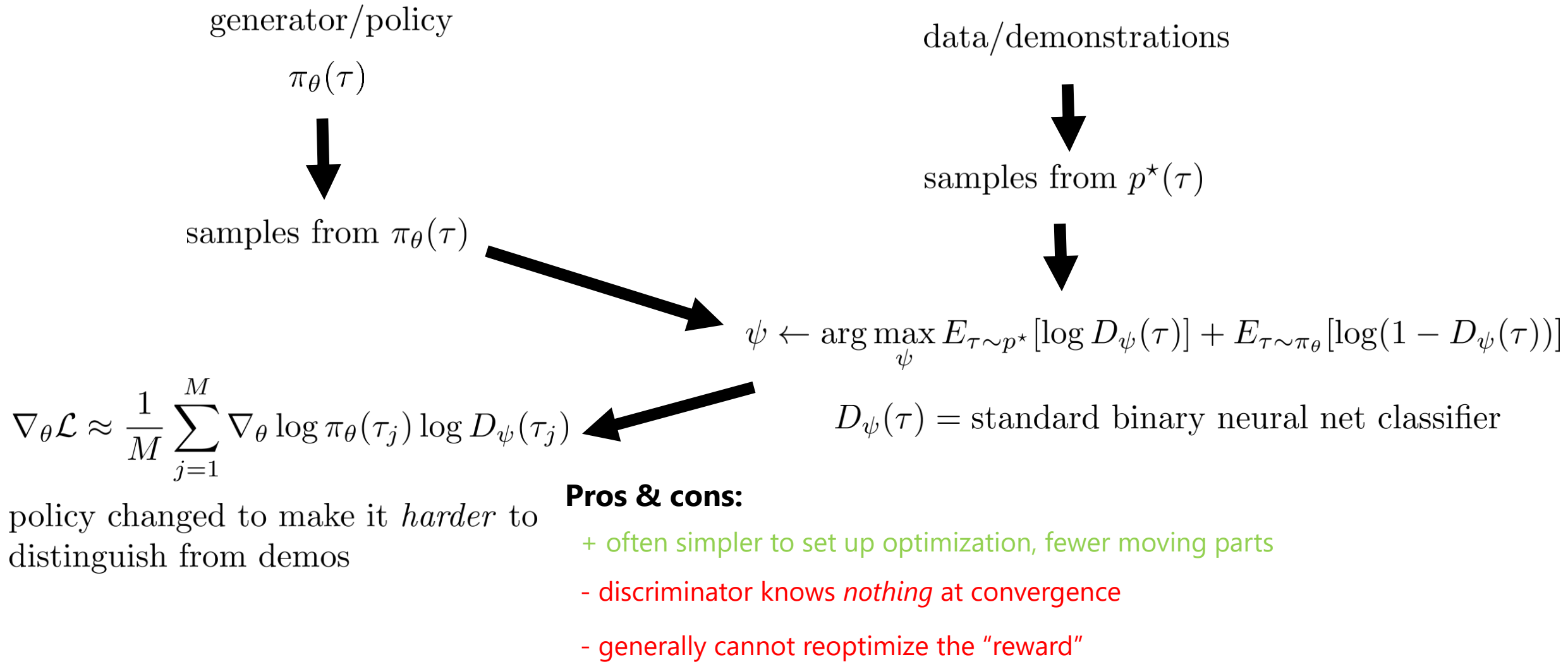
optimize this w.r.t.  $\psi$

$$\psi \leftarrow \arg \max_{\psi} E_{\tau \sim p^*} [\log D_\psi(\tau)] + E_{\tau \sim \pi_\theta} [\log(1 - D_\psi(\tau))]$$

we don't need importance weights anymore – they are subsumed into  $Z$



# Can we just use a regular discriminator?



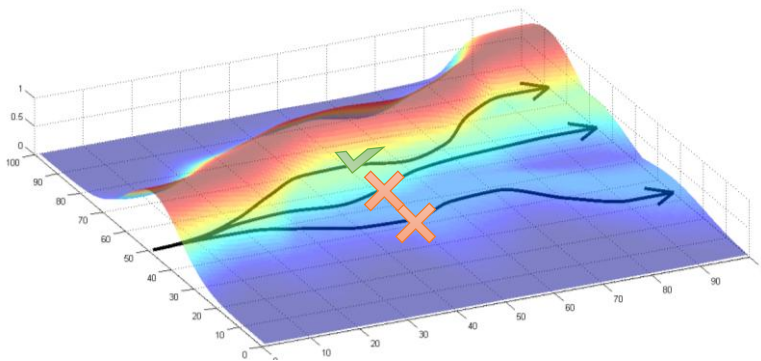
# Full RL from human feedback algorithm

RL from human feedback:

1. sample 2 (or more) trajectories  $\tau_i \sim \pi_\theta(\tau)$
2. ask a person to specify preferences  $\{\tau_i \succ \tau_j\}_{i,j}$
3. train reward  $\psi \leftarrow \arg \max_\psi \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$
4. train policy  $\pi_\theta$  with reward  $r_\psi$

this looks a lot like logistic regression

this can be incomplete optimization



$$p(\tau_i \succ \tau_j) = \frac{\exp \left( \sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)}{\exp \left( \sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right) + \exp \left( \sum_t r_\psi(\mathbf{s}_t^{(j)}, \mathbf{a}_t^{(j)}) \right)}$$

preference  $\tau_i \succ \tau_j$  means “I like  $i$  more than  $j$ ”

Intermission

# Part 7:

## Model-based RL

# Distributional shift in model-based RL



1. Get some data  $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$  using  $\pi_\beta$

2. Learn model  $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

3. Improve the policy “a bit” under  $f$



3. Set  $\pi_\beta \leftarrow \pi_f$

1. don't change the policy too much

$D_{\text{KL}}(\pi_f \| \pi_\beta) \leq \epsilon$       e.g., “trust region”

2. stick to places where the model is “confident”

- Use probabilistic, uncertainty-aware model
- Use penalty (“pessimism”) in the face of model uncertainty



1. Get some data  $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$  using  $\pi_\beta$

2. Learn **probabilistic** model  $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$

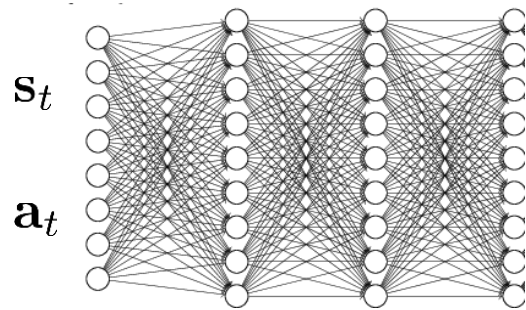
3. Get best policy  $\pi_p$  **in expectation** under  $p$



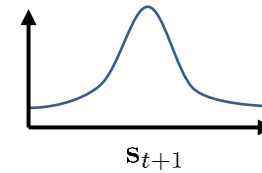
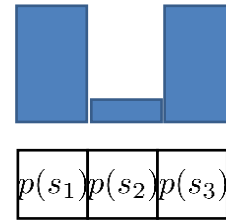
3. Set  $\pi_\beta \leftarrow \pi_p$

# How can we have uncertainty-aware models?

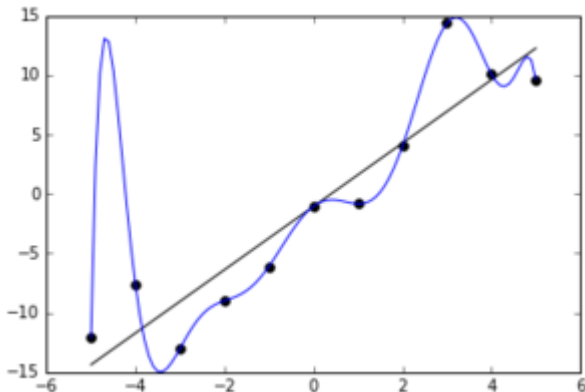
Idea 1: use output entropy



$$p(s_{t+1} | s_t, a_t)$$



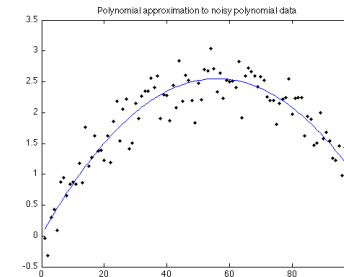
why is this not enough?



Two types of uncertainty:

*aleatoric* or *statistical* uncertainty →

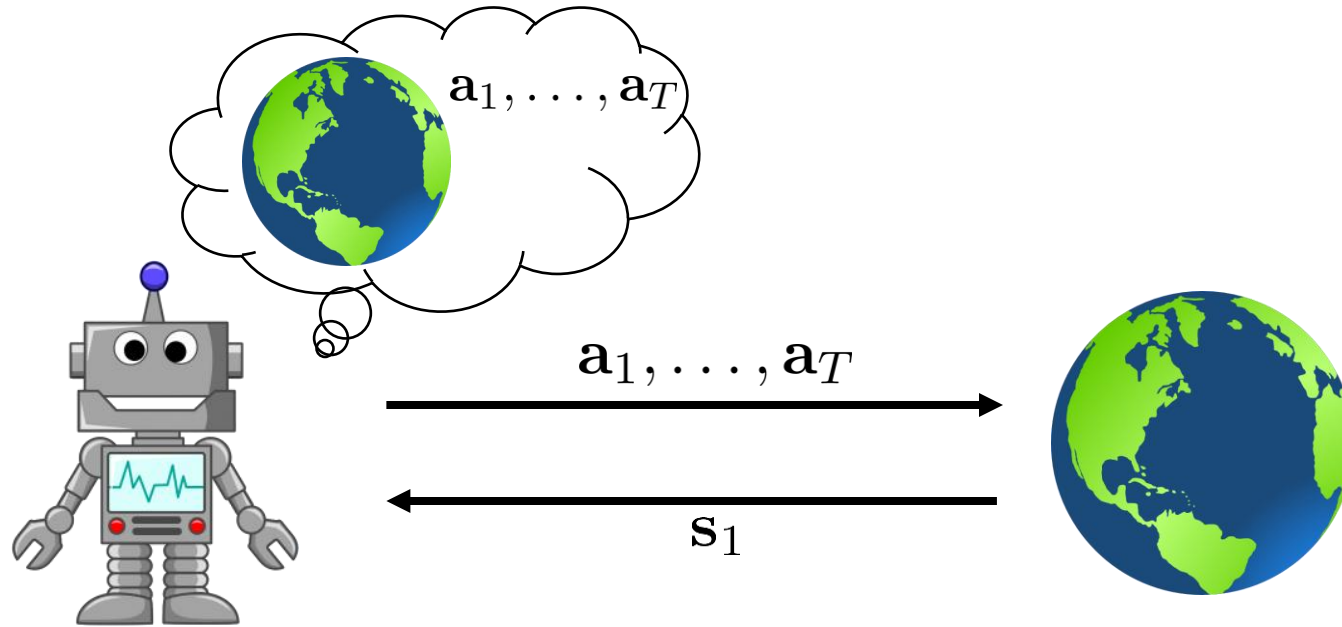
← *epistemic* or *model* uncertainty



*"the model is certain about the data, but we are not certain about the model"*

what is the variance here?

# The stochastic open-loop case

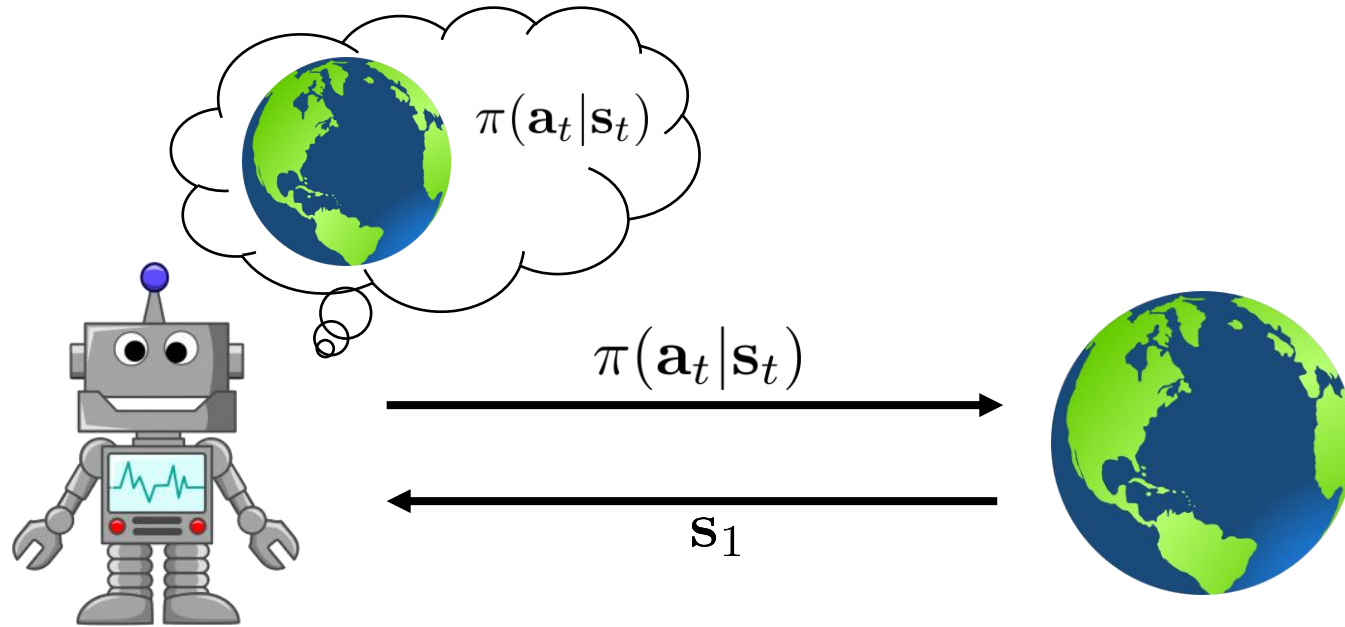


$$p_{\theta}(\mathbf{s}_1, \dots, \mathbf{s}_T | \mathbf{a}_1, \dots, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} E \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) | \mathbf{a}_1, \dots, \mathbf{a}_T \right]$$

why is this suboptimal?

# The stochastic **closed**-loop case



$$p(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T \pi(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\pi = \arg \max_{\pi} E_{\tau \sim p(\tau)} \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$



# How to plan with uncertainty

Before:  $J(\mathbf{a}_1, \dots, \mathbf{a}_H) = \sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t)$ , where  $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$

Now:  $J(\mathbf{a}_1, \dots, \mathbf{a}_H) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H r(\mathbf{s}_{t,i}, \mathbf{a}_t)$ , where  $\mathbf{s}_{t+1,i} = f_i(\mathbf{s}_{t,i}, \mathbf{a}_t)$

↖  
distribution over  
deterministic models

In general, for candidate action sequence  $\mathbf{a}_1, \dots, \mathbf{a}_H$ :

Step 1: sample  $\theta \sim p(\theta|\mathcal{D})$

Step 2: at each time step  $t$ , sample  $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta)$

Step 3: calculate  $R = \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$

Step 4: repeat steps 1 to 3 and accumulate the average reward

**Other options:** moment matching, more complex posterior estimation with BNNs, etc.

# Model-Based Acceleration (MBA)

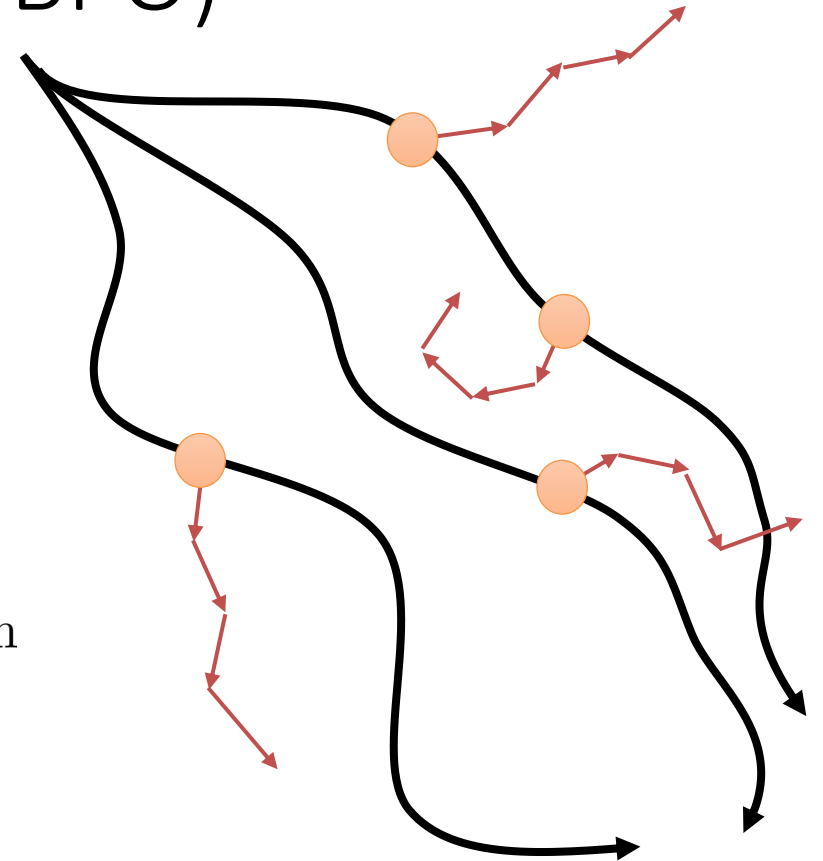
## Model-Based Value Expansion (MVE)

## Model-Based Policy Optimization (MBPO)

1. take some action  $\mathbf{a}_i$  and observe  $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ , add it to  $\mathcal{B}$
2. sample mini-batch  $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$  from  $\mathcal{B}$  uniformly
3. use  $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$  to update model  $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$
4. sample  $\{\mathbf{s}_j\}$  from  $\mathcal{B}$
5. for each  $\mathbf{s}_j$ , perform model-based rollout with  $\mathbf{a} = \pi(\mathbf{s})$
6. use all transitions  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$  along rollout to update Q-function

+ why is this a *good* idea?

- why is this a *bad* idea?

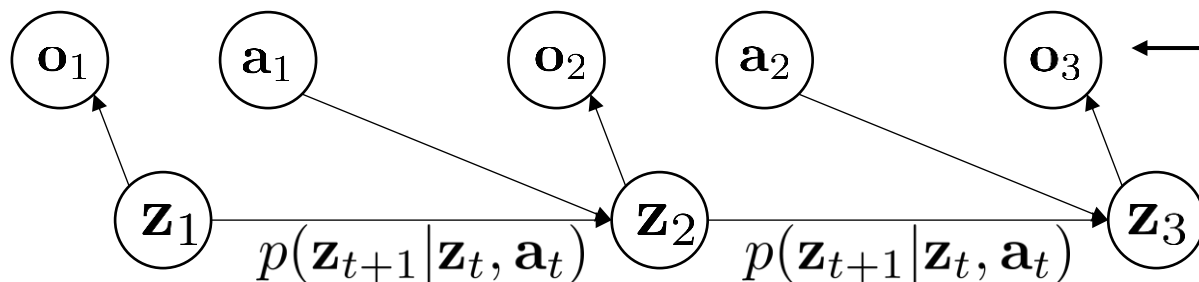
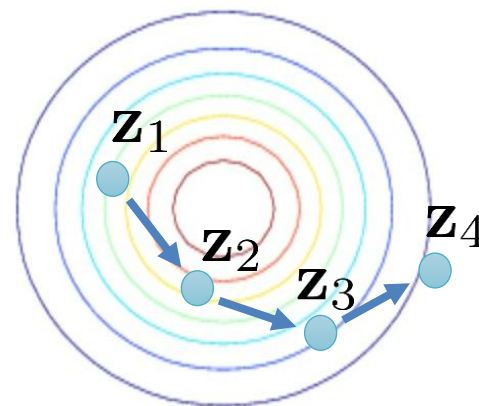
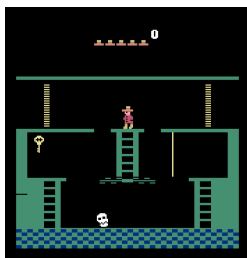
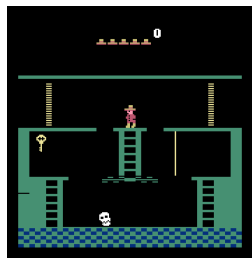
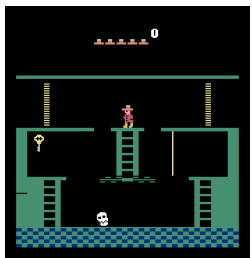


Gu et al. Continuous deep Q-learning with model-based acceleration. '16

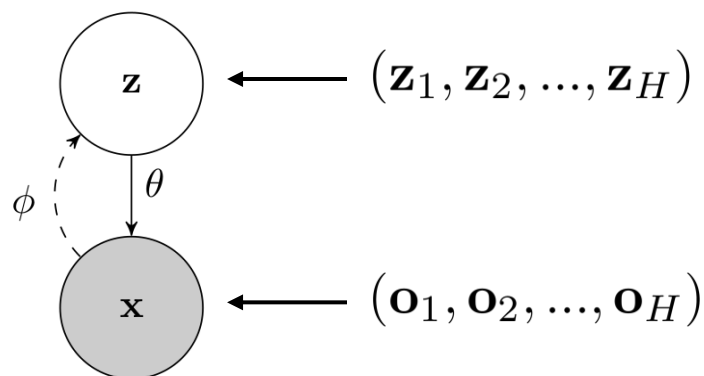
Feinberg et al. Model-based value expansion. '18

Janner et al. When to trust your model: model-based policy optimization. '19

# Latent state space models



we are not in partially observed setting



What is the **prior**?

What is the **decoder**?

What is the **encoder**?

Diagram illustrating the prior distribution  $\mathcal{N}(0, \mathbf{I})$  and the learned transition probabilities  $p(z_{t+1} | z_t, a_t)$ .

$$p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$$

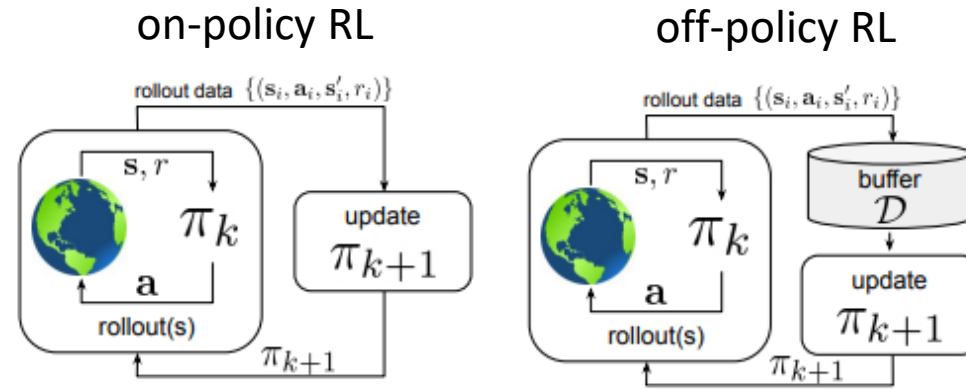
$$p_{\theta}(\mathbf{o} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{z}_t)$$

$$q_{\phi}(\mathbf{z} | \mathbf{o}) = \prod_t q_{\phi}(\mathbf{z}_t | \mathbf{o}_{1:t})$$

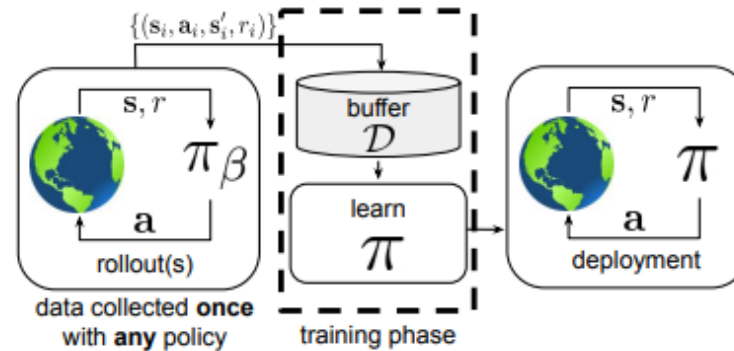
# Part 8:

## Offline RL

# Offline reinforcement learning



## offline reinforcement learning



# Where does it appear for each off-policy RL method?

Q-learning & Q-function actor-critic

$$\phi \leftarrow \arg \min_{\phi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \pi_{\beta}(\mathbf{s}, \mathbf{a})} \|Q_{\phi}^{\pi_{\theta}}(\mathbf{s}, \mathbf{a}) - y\|^2$$

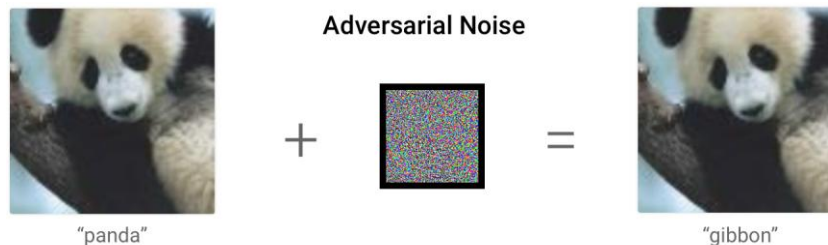
$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}' | \mathbf{s}'_i)} [Q_{\phi}^{\pi_{\theta}}(\mathbf{s}'_i, \mathbf{a}')] ]$$

because we only have  $\mathbf{s}'_i$  for  $\mathbf{s}_i, \mathbf{a}_i$

$$(\mathbf{s}_i, \mathbf{a}_i) \sim \pi_{\beta}(\mathbf{s}, \mathbf{a})$$

$$\pi_{\theta}(\mathbf{a} | \mathbf{s}) \neq \pi_{\beta}(\mathbf{a} | \mathbf{s})$$

$$\text{worse: } \theta \leftarrow \arg \max_{\theta} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s})} [Q_{\phi}^{\pi_{\theta}}(\mathbf{s}, \mathbf{a})]$$



# Where does it appear for each off-policy RL method?

model-based RL

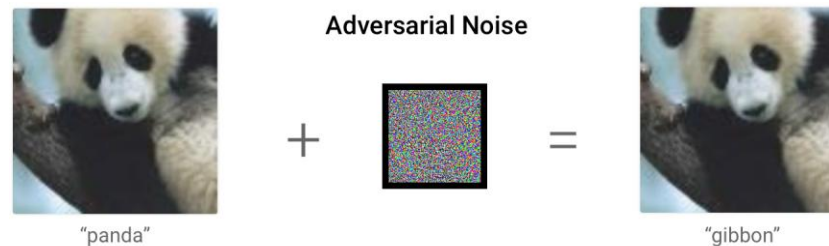
$$f \leftarrow \arg \min_f \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \pi_\beta(\mathbf{s}, \mathbf{a}), \mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \mathbf{a})} \|f(\mathbf{s}, \mathbf{a}) - \mathbf{s}'\|^2$$

$\mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \pi_\beta(\mathbf{s}, \mathbf{a}), \mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \mathbf{a})} \|f(\mathbf{s}, \mathbf{a}) - \mathbf{s}'\|^2$  is low

$\mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \pi_\theta(\mathbf{s}, \mathbf{a}), \mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \mathbf{a})} \|f(\mathbf{s}, \mathbf{a}) - \mathbf{s}'\|^2$  is not, for general  $\pi_\theta \neq \pi_\beta$

worse:

we pick  $\pi_\theta$  to *maximize* the reward under  $f$ !



# Where does it appear for each off-policy RL method?

off-policy policy gradient (importance-sampled surrogate):

$$J(\theta) = \sum_{i=1}^N \sum_{t=1}^H \underbrace{\frac{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\beta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}}_{\text{ratio very far from 1.0 = exploding variance}} A^{\pi_{\theta}}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \underbrace{\beta \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}_{\text{"just don't go there"}}$$

This is an example of a  
*policy constraint*

the answer to counterfactual questions has “high variance” (depends on whether you saw it or not!)



# Some principles for offline RL

- Many different methods, similar principles seem to be effective:

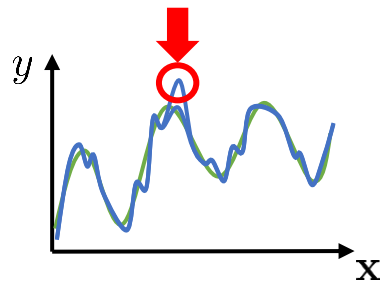
use value-based methods (i.e., Q-learning or Q-function actor-critic)

somehow fix the distributional shift problem

$$D_{\text{KL}}(\pi(\mathbf{a}|\mathbf{s})||\pi_{\beta}(\mathbf{a}|\mathbf{s})) \leq \epsilon$$

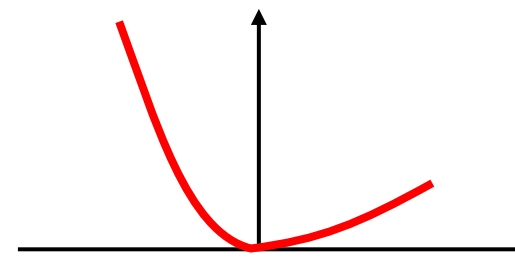
policy constraints

train the policy so that it  
stays close to data



pessimism (e.g., CQL)

train the Q-function so that  
it doesn't overestimate



avoid OOD actions in updates  
(e.g., IQL)

set up the whole algorithm to  
avoid out-of-sample actions

# Explicit policy constraint methods

How do we implement constraints?

1. Modify the actor objective

$$\theta \leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a})]$$

Lagrange multiplier

easy to compute and differentiate  
for Gaussian or categorical policies

1a

$$\theta \leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a}) + \lambda \log \pi_{\beta}(\mathbf{a}|\mathbf{s}_i)] + \lambda \mathcal{H}(\pi_{\theta}(\mathbf{a}|\mathbf{s}_i))$$
$$D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\beta}) = E_{\pi_{\theta}}[\log \pi_{\theta}(\mathbf{a}|\mathbf{s}) - \log \pi_{\beta}(\mathbf{a}|\mathbf{s})] = -E_{\pi_{\theta}}[\log \pi_{\beta}(\mathbf{a}|\mathbf{s})] - \mathcal{H}(\pi_{\theta}) \quad \text{“reverse KL”}$$

1b

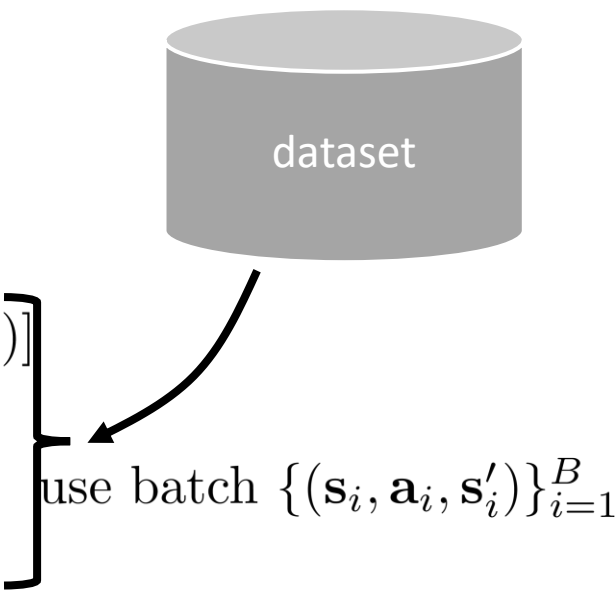
$$\theta \leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a})] + \lambda \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i)$$
$$D_{\text{KL}}(\pi_{\beta} \parallel \pi_{\theta}) = -E_{\pi_{\beta}}[\log \pi_{\theta}(\mathbf{a}|\mathbf{s})] + \text{const} \quad \text{“forward KL”}$$

sometimes called *[method]+BC* (e.g., DDPG+BC, TD3+BC, SAC+BC, etc.)

generally not the *best* way to fix the problem, but can work well if done right

# Example: BRAC-like algorithm

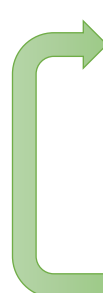
offline actor-critic algorithm (BRAC-like):

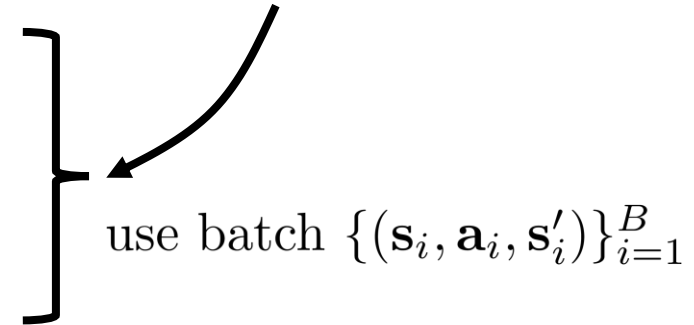
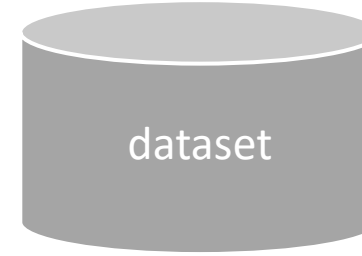
- 
1. evaluate  $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}') - \lambda D(\pi_\theta(\cdot|\mathbf{s}'_i), \pi_\beta(\cdot, \mathbf{s}'_i))]$
  2. update  $\phi$  using  $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
  3.  $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})] - \lambda D(\pi_\theta(\cdot|\mathbf{s}_i), \pi_\beta(\cdot|\mathbf{s}_i))$
  4.  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- 

for example  $D(\pi_\theta, \pi_\beta) = D_{\text{KL}}(\pi_\theta \| \pi_\beta)$

# Example: AC+BC-like algorithm

offline actor-critic algorithm (AC+BC-like):

- 
1. evaluate  $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' | \mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')] ]$
  2. update  $\phi$  using  $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
  3.  $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})] + \lambda \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i)$
  4.  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



# Implicit policy constraint methods

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg \max_{\pi} E_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D_{\text{KL}}(\pi \parallel \pi_{\beta}) \leq \epsilon$$

$$\pi^*(\mathbf{a}|\mathbf{s}) = \frac{1}{Z(\mathbf{s})} \pi_{\beta}(\mathbf{a}|\mathbf{s}) \exp \left( \frac{1}{\lambda} A^{\pi}(\mathbf{s}, \mathbf{a}) \right)$$

straightforward to show via duality

**See also:**

Peters et al. (REPS)

Rawlik et al. (“psi-learning”)

...many follow-ups

approximate via **weighted** max likelihood!

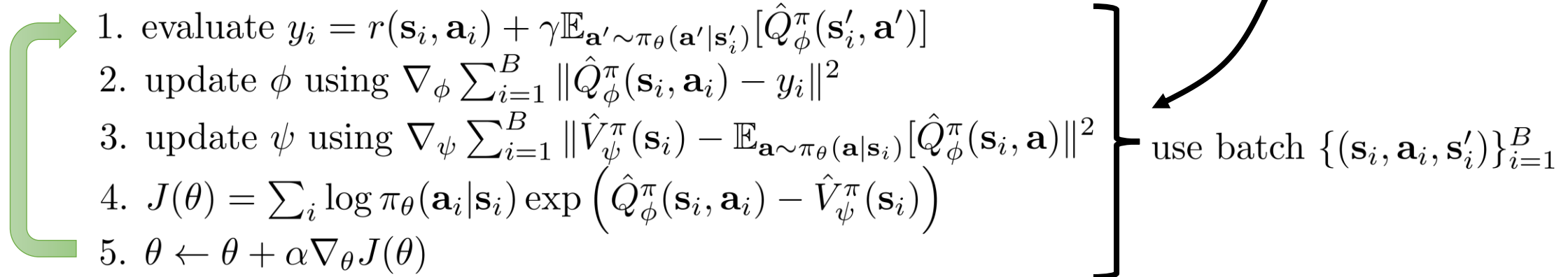
$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg \max_{\pi} E_{(\mathbf{s}, \mathbf{a}) \sim \pi_{\beta}} \left[ \log \pi(\mathbf{a}|\mathbf{s}) \overbrace{\frac{1}{Z(\mathbf{s})} \exp \left( \frac{1}{\lambda} A^{\pi_{\text{old}}}(\mathbf{s}, \mathbf{a}) \right)}^{w(\mathbf{s}, \mathbf{a})} \right]$$

↑  
samples from dataset  
 $\mathbf{a} \sim \pi_{\beta}(\mathbf{a}|\mathbf{s})$

↖  
critic can be used  
to give us this

# Example: AWAC-like algorithm

offline actor-critic algorithm (AWAC-like):

- 
1. evaluate  $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')] ]$
  2. update  $\phi$  using  $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
  3. update  $\psi$  using  $\nabla_\psi \sum_{i=1}^B \|\hat{V}_\psi^\pi(\mathbf{s}_i) - \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]\|^2$
  4.  $J(\theta) = \sum_i \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \exp \left( \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi^\pi(\mathbf{s}_i) \right)$
  5.  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

can also just approximate  $\hat{V}^\pi(\mathbf{s}_i) \approx \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})$  where  $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)$

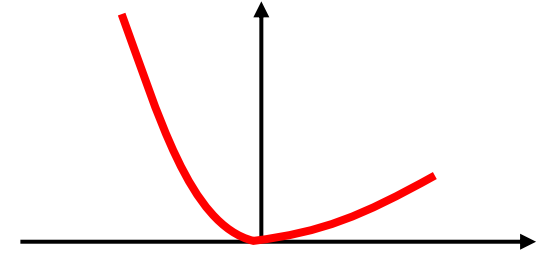
what are the *advantages* of this approach?

what are possible *problems* with this approach?

# Implicit Q-learning (IQL)

Q-learning with *implicit* policy improvement

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + V(\mathbf{s}') \quad V \leftarrow \arg \min_V \frac{1}{N} \sum_{i=1}^N \ell_2^\tau(V(\mathbf{s}_i), Q(\mathbf{s}_i, \mathbf{a}_i))$$



$$V(\mathbf{s}) \leftarrow \max_{\mathbf{a} \in \Omega(\mathbf{s})} Q(\mathbf{s}, \mathbf{a})$$

$$\Omega(\mathbf{s}) = \{\mathbf{a} : \pi_\beta(\mathbf{a}|\mathbf{s}) \geq \epsilon\}$$



$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \max_{\mathbf{a}' \in \Omega(\mathbf{s}')} Q(\mathbf{s}', \mathbf{a}')$$

“implicit” policy

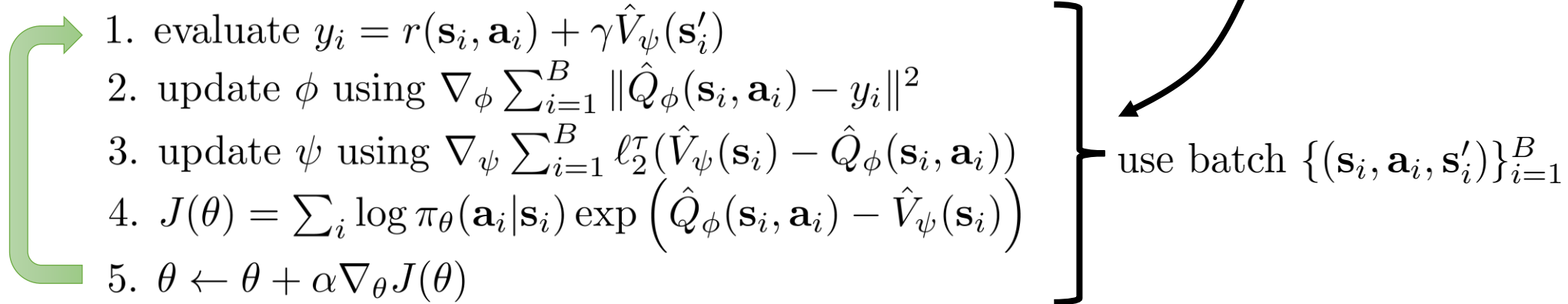
if we use  $\ell_2^\tau$  for big  $\tau$

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \delta(\mathbf{a} = \arg \max_{\mathbf{a} \in \Omega(\mathbf{s})} Q(\mathbf{s}, \mathbf{a}))$$

Now we can do value function updates without ever risking out-of-distribution actions!

# Implicit Q-learning (IQL)

offline actor-critic with IQL:

- 
1. evaluate  $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\psi(\mathbf{s}'_i)$
  2. update  $\phi$  using  $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
  3. update  $\psi$  using  $\nabla_\psi \sum_{i=1}^B \ell_2^\tau(\hat{V}_\psi(\mathbf{s}_i) - \hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i))$
  4.  $J(\theta) = \sum_i \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i) \exp(\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi(\mathbf{s}_i))$
  5.  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- use batch  $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^B$

there are no expectations under  $\pi_\theta$  anywhere

critic is also completely independent of actor

hence implicit “Q-learning,” not really “actor-critic”



# Conservative Q-learning (CQL)

A *better* bound:

$$\hat{Q}^\pi = \arg \min_Q \max_\mu \left\{ \begin{array}{l} \text{always pushes Q-values down} \quad \text{push up on } (\mathbf{s}, \mathbf{a}) \text{ samples in data} \\ \alpha E_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \alpha E_{(\mathbf{s}, \mathbf{a}) \sim D} [Q(\mathbf{s}, \mathbf{a})] \\ + E_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} \left[ (Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + E_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2 \right] \end{array} \right\} \mathcal{L}_{\text{CQL}}(\hat{Q}^\pi)$$

no longer guaranteed that  $\hat{Q}^\pi(\mathbf{s}, \mathbf{a}) \leq Q^\pi(\mathbf{s}, \mathbf{a})$  for all  $(\mathbf{s}, \mathbf{a})$

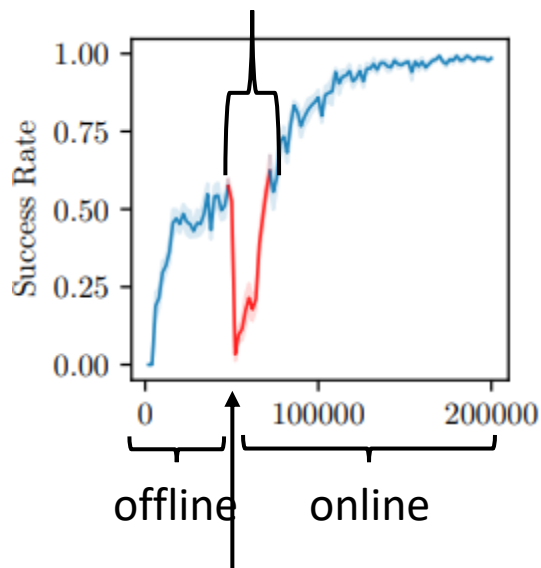
but guaranteed that  $E_{\pi(\mathbf{a}|\mathbf{s})}[\hat{Q}^\pi(\mathbf{s}, \mathbf{a})] \leq E_{\pi(\mathbf{a}|\mathbf{s})}[Q^\pi(\mathbf{s}, \mathbf{a})]$  for all  $\mathbf{s} \in D$

# What's the problem with offline to online RL?

**Example:** offline to online RL  
with CQL

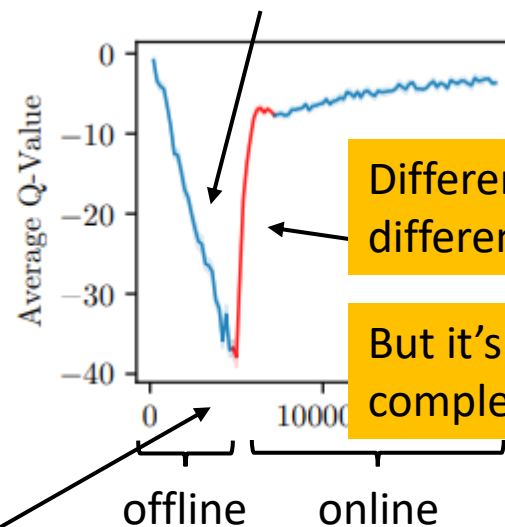
$$\hat{Q}^{\pi} = \arg \min_Q \max_{\pi} \alpha E_{\mathbf{s} \sim D, \mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \alpha E_{\mathbf{s}, \mathbf{a} \sim D} [Q(\mathbf{s}, \mathbf{a})] \\ + E_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} \left[ (Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + E_{\pi} [Q(\mathbf{s}', \mathbf{a}')]))^2 \right]$$

this period wasted recovering  
offline performance



online training starts here (at 50k steps)

underestimating too much here



Different offline RL algorithms have  
different versions of this problem

But it's very hard to fix this  
completely!

# IDQL: simple offline-to-online RL with diffusion

1. train  $\hat{Q}_\phi(\mathbf{s}, \mathbf{a})$  using IQL (without any actor)  $\left\{ \begin{array}{l} 1. \text{ update } \phi \text{ using } \nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2 \\ 2. \text{ update } \psi \text{ using } \nabla_\psi \sum_{i=1}^B \ell_2^\tau(\hat{V}_\psi(\mathbf{s}_i) - \hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i)) \\ 3. J(\theta) = \sum_i \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i) \exp(\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi(\mathbf{s}_i)) \end{array} \right.$
2. train  $\pi_\theta(\mathbf{a}|\mathbf{s})$  as a diffusion/flow model with BC  $\pi_\theta(\mathbf{a}|\mathbf{s}) \approx \pi_\beta(\mathbf{a}|\mathbf{s})$
3. at test time, sample  $\{\mathbf{a}_1, \dots, \mathbf{a}_K\}$  from  $\pi_\theta(\mathbf{a}|\mathbf{s})$  at each step  
pick  $\arg \max_{\mathbf{a}_k} \hat{Q}_\phi(\mathbf{s}, \mathbf{a}_k)$  no OOD actions because  $\pi_\theta(\mathbf{a}|\mathbf{s}) \approx \pi_\beta(\mathbf{a}|\mathbf{s})$

why is this good?

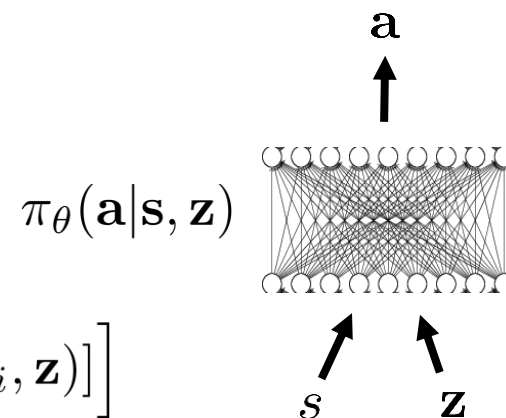
why is this bad?

# FQL: actor that stays “close” to diffusion model

1. train  $\pi_{\text{flow}}(\mathbf{a}|\mathbf{s}, \mathbf{z})$  as a diffusion/flow model with BC
2. run offline actor-critic with a special BC regularizer

$$\pi_{\text{flow}}(\mathbf{a}|\mathbf{s}) \approx \pi_{\beta}(\mathbf{a}|\mathbf{s})$$

this actor is **not** a diffusion model! It's just a regular neural net that takes in  $\mathbf{z}$



$$J(\theta) = \sum_i \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \left[ \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i, \mathbf{z})} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})] + \mathbb{E}_{\mathbf{a} \sim \pi_{\text{flow}}(\mathbf{a}|\mathbf{s}, \mathbf{z})} [\lambda \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i, \mathbf{z})] \right]$$

clone the action  $\pi_{\text{flow}}$  would take *for that value of  $\mathbf{z}$*

why is this good?

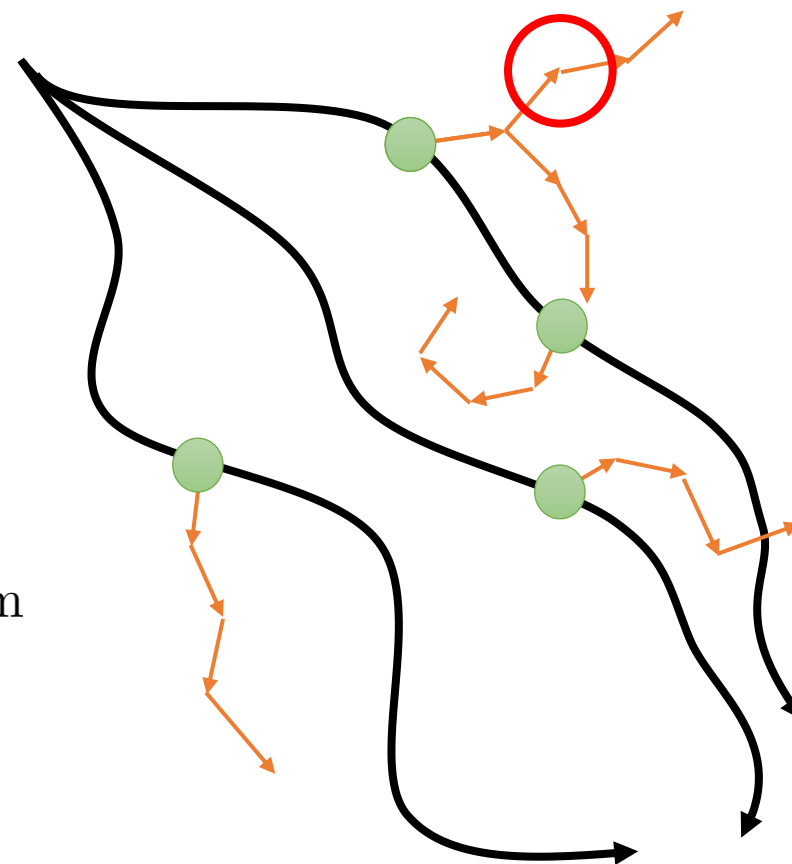
# MOPO: Model-Based Offline Policy Optimization

solution: “punish” the policy for exploiting

$$\tilde{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) - \lambda u(\mathbf{s}, \mathbf{a})$$

← uncertainty penalty

...and then use any existing model-based RL algorithm



# Part 9: Exploration

# Optimistic exploration in RL

UCB: 
$$a = \arg \max \hat{\mu}_a + \underbrace{\sqrt{\frac{2 \ln H}{N(a)}}}_{\text{“exploration bonus”}}$$

lots of functions work, so long as they decrease with  $N(a)$

can we use this idea with MDPs?

count-based exploration: use  $N(\mathbf{s}, \mathbf{a})$  or  $N(\mathbf{s})$  to add *exploration bonus*

use  $r^+(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \mathcal{B}(N(\mathbf{s}))$



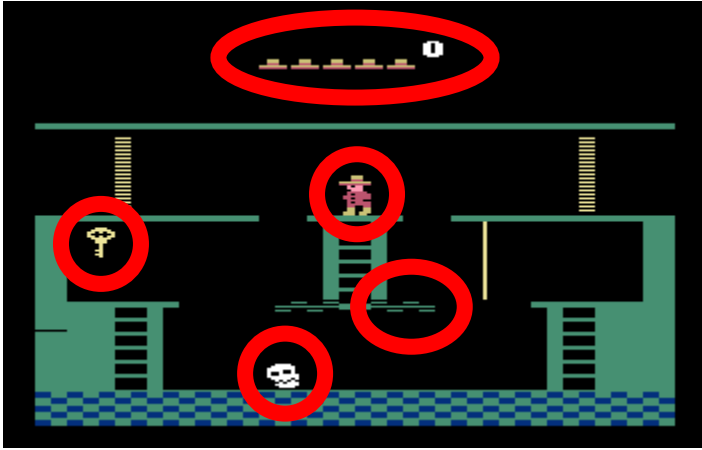
bonus that decreases with  $N(\mathbf{s})$

use  $r^+(\mathbf{s}, \mathbf{a})$  instead of  $r(\mathbf{s}, \mathbf{a})$  with any model-free algorithm

+ simple addition to any RL algorithm

- need to tune bonus weight

# Exploring with pseudo-counts



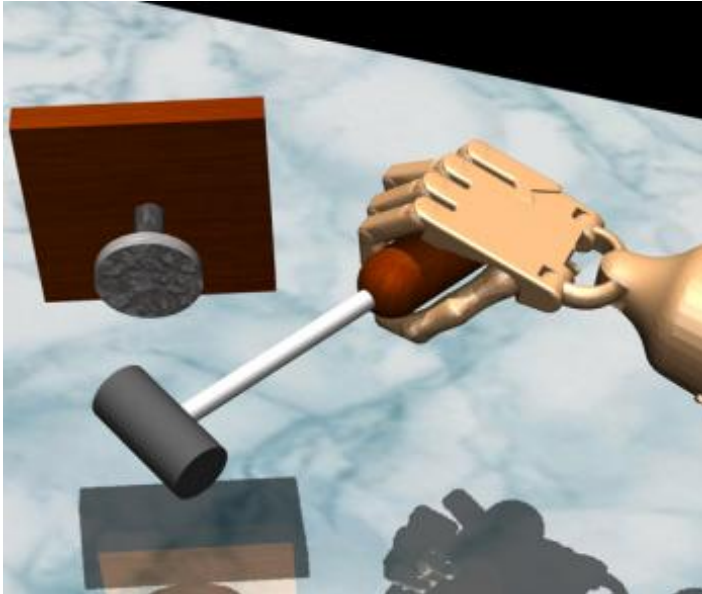
fit model  $p_{\theta}(\mathbf{s})$  to all states  $\mathcal{D}$  seen so far

take a step  $i$  and observe  $\mathbf{s}_i$

fit new model  $p_{\theta'}(\mathbf{s})$  to  $\mathcal{D} \cup \mathbf{s}_i$

use  $p_{\theta}(\mathbf{s}_i)$  and  $p_{\theta'}(\mathbf{s}_i)$  to estimate  $\hat{N}(\mathbf{s})$

set  $r_i^+ = r_i + \mathcal{B}(\hat{N}(\mathbf{s}))$  ← “pseudo-count”



how to get  $\hat{N}(\mathbf{s})$ ? use the equations

$$p_{\theta}(\mathbf{s}_i) = \frac{\hat{N}(\mathbf{s}_i)}{\hat{n}}$$

$$p_{\theta'}(\mathbf{s}_i) = \frac{\hat{N}(\mathbf{s}_i) + 1}{\hat{n} + 1}$$

two equations and two unknowns!

$$\hat{N}(\mathbf{s}_i) = \hat{n} p_{\theta}(\mathbf{s}_i) \quad \hat{n} = \frac{1 - p_{\theta'}(\mathbf{s}_i)}{p_{\theta'}(\mathbf{s}_i) - p_{\theta}(\mathbf{s}_i)} p_{\theta}(\mathbf{s}_i)$$



# Heuristic estimation of counts via errors

let's say we have some **target** function  $f^*(\mathbf{s}, \mathbf{a})$

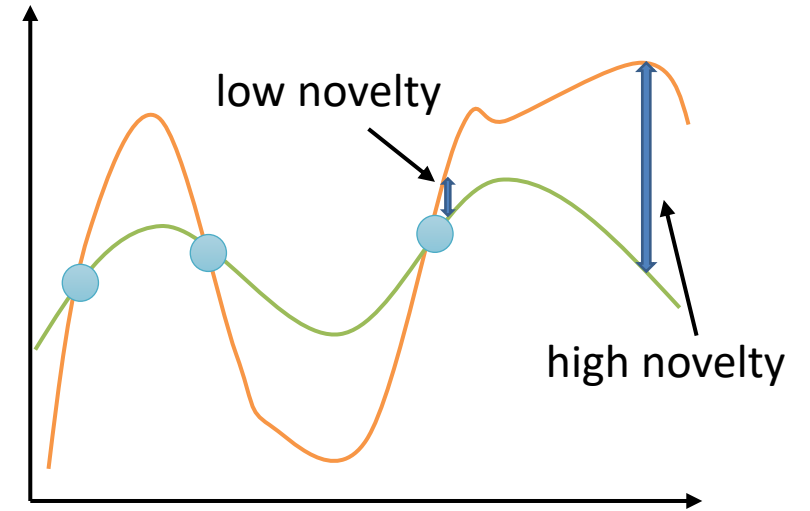
given our buffer  $\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i)\}$ , fit  $\hat{f}_\theta(\mathbf{s}, \mathbf{a})$

use  $\mathcal{E}(\mathbf{s}, \mathbf{a}) = \|\hat{f}_\theta(\mathbf{s}, \mathbf{a}) - f^*(\mathbf{s}, \mathbf{a})\|^2$  as bonus

what should we use for  $f^*(\mathbf{s}, \mathbf{a})$ ?

one common choice: set  $f^*(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$  – i.e., next state prediction

even simpler:  $f^*(\mathbf{s}, \mathbf{a}) = f_\phi(\mathbf{s}, \mathbf{a})$ , where  $\phi$  is a *random* parameter vector



# Part 10:

## RL theory

# Let's prove that value iteration converges

value iteration:  $V \leftarrow \max_a [r + \gamma PV]$

$$TV = \max_a [r + \gamma PV]$$

“Bellman optimality operator”

$$|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|$$

$$\begin{aligned} \|TV - TU\|_\infty &\leq |TV(s) - TU(s)| = |\max_a [r(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s')] - \max_a [r(s, a) + \gamma \sum_{s'} P(s'|s, a)U(s')]| \\ &\leq \max_a |r(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s') - r(s, a) - \gamma \sum_{s'} P(s'|s, a)U(s')| \\ &= \gamma \max_a |\sum_{s'} P(s'|s, a)(V(s') - U(s'))| \\ &\leq \gamma \max_{s'} |V(s') - U(s')| \\ &= \gamma \|V - U\|_\infty \end{aligned}$$

# The setup

“oracle exploration”: for every  $(s, a)$ , sample  $s' \sim P(s'|s, a)$   $N$  times

simple “model based” algorithm:

1.  $\hat{P}(s'|s, a) = \frac{\#(s, a, s')}{N}$
2. Given  $\pi$ , use  $\hat{P}$  to estimate  $\hat{Q}^\pi$

how close is  $\hat{Q}^\pi$  to  $Q^\pi$ ? ←

$$\|Q^\pi(s, a) - \hat{Q}^\pi(s, a)\|_\infty \leq \epsilon \quad \begin{array}{l} \text{with probability at least } 1 - \delta \\ \text{if } N \geq f(\epsilon, \delta) \end{array}$$

$$\max_{s, a} |Q^\pi(s, a) - \hat{Q}^\pi(s, a)| \leq \epsilon$$

good to use  $\|\cdot\|_\infty$  if we want *worst-case* performance

how close is  $\hat{Q}^*$  if we learn it using  $\hat{P}$ ? ←

$$\|Q^*(s, a) - \hat{Q}^*(s, a)\|_\infty \leq \epsilon$$

optimal Q-function learned under  $\hat{P}$

how good is the resulting policy? ←

$$\|Q^*(s, a) - Q^{\hat{\pi}}(s, a)\|_\infty \leq \epsilon$$

the arg max policy corresponding to that Q-function

# What does this mean?

$$\|Q^\pi - \hat{Q}^\pi\|_\infty \leq \frac{\gamma}{(1-\gamma)^2} c_2 \sqrt{\frac{|S| \log 1/\delta}{N}}$$

error grows **quadratically** in the horizon  
each backup “accumulates” error

more samples = lower error

# Some simple implications...

$$\|Q^\pi - \hat{Q}^\pi\|_\infty \leq \epsilon \qquad \epsilon = \frac{\gamma}{(1-\gamma)^2} c_2 \sqrt{\frac{|S| \log 1/\delta}{N}}$$

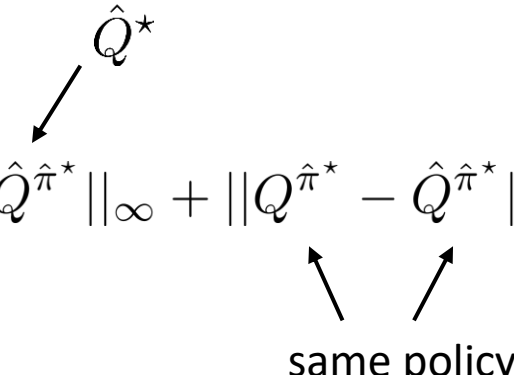
what about  $\|Q^* - \hat{Q}^*\|_\infty$ ?

$$|\sup_x f(x) - \sup_x g(x)| \leq \sup_x |f(x) - g(x)|$$

$$\|Q^* - \hat{Q}^*\|_\infty = \|\sup_\pi Q^\pi - \sup_\pi \hat{Q}^\pi\|_\infty \leq \sup_\pi \|Q^\pi - \hat{Q}^\pi\|_\infty \leq \epsilon$$

what about  $\|Q^* - Q^{\hat{\pi}^*}\|_\infty$ ?

$$\|Q^* - Q^{\hat{\pi}^*}\|_\infty = \|Q^* - \hat{Q}^{\hat{\pi}^*} + \hat{Q}^{\hat{\pi}^*} - Q^{\hat{\pi}^*}\|_\infty \leq \|Q^* - \hat{Q}^{\hat{\pi}^*}\|_\infty + \|Q^{\hat{\pi}^*} - \hat{Q}^{\hat{\pi}^*}\|_\infty \leq 2\epsilon$$

  
same policy

# Analyzing fitted Q-iteration

abstract model of exact Q-iteration:

$$Q_{k+1} \leftarrow TQ_k$$

Bellman operator  
 $TQ = r + \gamma P \max_a Q$

abstract model of approximate fitted Q-iteration:

$$\hat{Q}_{k+1} \leftarrow \arg \min_{\hat{Q}} \|\hat{Q} - \hat{T}\hat{Q}_k\|$$

which norm?  
 no convergence if  $\|\cdot\|_2$   
 we'll assume  $\|\cdot\|_\infty$

Question: as  $k \rightarrow \infty$ ,  $\hat{Q}_k \rightarrow ?$

$$\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_\infty \leq ?$$

where do errors come from?

$$T \neq \hat{T} \quad \text{"sampling error"}$$

$$\hat{Q}_{k+1} \neq \hat{T}\hat{Q}_k \quad \text{"approximation error"}$$

approximate Bellman operator

$$\hat{T}Q = \hat{r} + \gamma \hat{P} \max_a Q$$

$$\hat{r}(s, a) = \frac{1}{N(s, a)} \sum_i \delta((s_i, a_i) = (s, a)) r_i$$

$$\hat{P}(s'|s, a) = \frac{N(s, a, s')}{N(s, a)}$$

**Note:** these are **not** models, this is the effect of averaging together transitions in the data!

# Putting it together

$$\|\hat{T}Q - TQ\|_\infty \leq 2R_{\max}c_1\sqrt{\frac{\log |S||A|/\delta}{2N}} + c_2\|Q\|_\infty\sqrt{\frac{\log |S|/\delta}{N}} \quad \text{“sampling error”}$$

$$\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_\infty \leq \frac{1}{1-\gamma} \max_k \epsilon_k = \frac{1}{1-\gamma} \max_k \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty \quad \text{“approximation error”}$$

how much  $\hat{Q}_{k+1}$  differs from  $T\hat{Q}_k$   
 due to: sampling error ( $T \neq \hat{T}$ )  
 approximation error ( $\hat{Q}_k \neq \hat{T}\hat{Q}_{k-1}$ )

$$\begin{aligned} \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty &= \|\hat{Q}_k - \hat{T}\hat{Q}_{k-1} + \hat{T}\hat{Q}_{k-1} - T\hat{Q}_{k-1}\|_\infty \\ &\leq \underbrace{\|\hat{Q}_k - \hat{T}\hat{Q}_{k-1}\|_\infty}_{\text{approximation}} + \underbrace{\|\hat{T}\hat{Q}_{k-1} - T\hat{Q}_{k-1}\|_\infty}_{\text{sampling}} \end{aligned}$$



# What does it all mean?

$$\|\hat{T}Q - TQ\|_\infty \leq 2R_{\max}c_1\sqrt{\frac{\log|S||A|/\delta}{2N}} + c_2\|Q\|_\infty\sqrt{\frac{\log|S|/\delta}{N}} \quad \text{“sampling error”}$$

$\mathcal{O}\left(R_{\max}\frac{1}{1-\gamma}\right)$

$$\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_\infty \leq \frac{1}{1-\gamma} \max_k \epsilon_k = \frac{1}{1-\gamma} \max_k \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty \quad \text{“approximation error”}$$

$$\|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty \leq \|\hat{Q}_k - \hat{T}\hat{Q}_{k-1}\|_\infty + \|\hat{T}\hat{Q}_{k-1} - T\hat{Q}_{k-1}\|_\infty$$

error “compounds” with horizon, over iterations and due to sampling

so far we needed strong (infinity norm) assumptions

more advanced results can be derived with p-norms under some distribution:

$$\|\hat{Q}_k - Q^*\|_{p,\mu} = \left( E_{(s,a) \sim \mu(s,a)} [\|\hat{Q}_k(s,a) - Q^*(s,a)\|^p] \right)^{1/p}$$